Final Exam

Graded

Student

Menger Wen

Total Points

214.5 / 222 pts

Question 1

Honor Code

1 / 1 pt

✔ **+ 1 pt** Correct: they actually copied out the code and signed it.

+ 0 pts Odd: They only signed the statement.

+ 0 pts They did not sign the statement. Only a minor error since they signed above.

− 5 pts They did not sign *and* did not sign above.

− 5 pts They did not copy the statement down completely.

− 10,000 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

+ 1 pt Said sorry and signed in a regrade request.

Question 2

Favorite Topic

4 / 4 pts

+ **0 pts** Blank
+ **0 pts** Something that is not a deep learning topic

Generative approaches

+ **4 pts** VAEs
+ **4 pts** Diffusion
+ **4 pts** GANs
+ **4 pts** GPT
+ **4 pts** Just generative generally

Basic architectures

+ **4 pts** CNN
+ **4 pts** GNNs
+ **4 pts** RNNs
+ ✔ **4 pts** Transformers
+ **4 pts** State-space models

Basic building blocks and phenomena

+ **4 pts** SGD
+ **4 pts** Residual connections
+ **4 pts** Variational inference
+ **4 pts** Attention
+ **4 pts** in-context learning
+ **4 pts** Alignment
+ **4 pts** Self-supervision
+ **4 pts** MAML
+ **4 pts** Ridge regression
+ **4 pts** Policy Gradients
+ **4 pts** Backprop
+ **4 pts** The idea of latent spaces
+ **4 pts** Random features

+ 4 pts Xavier and He initializations

+ 4 pts Dimensionality reduction

+ 4 pts Adam/momentum

+ 4 pts Data Augmentations

+ 4 pts Dropout

Fine-tuning

+ 4 pts General Fine-tuning

+ 4 pts LoRA

+ 4 pts Soft-prompts

+ 4 pts The idea of PEFT generally

+ 4 pts Vision

+ 4 pts AI Reliability: adversarial robustness and other issues

+ 4 pts Multimodal models

+ 4 pts LLMs

+ 4 pts Deep RL

+ 4 pts Beam search

+ 4 pts Self-organizing feature maps

+ 4 pts Speeding things up

+ 4 pts Representation learning

+ 4 pts Interpretability

+ 4 pts Meta Learning

+ 4 pts Transfer Learning

+ 4 pts Forgetting

+ 4 pts Scaling laws

+ 4 pts Neural Fields

+ 4 pts Test-time training

+ 4 pts Positional Encodings

+ 4 pts Optimizers

+ 4 pts Muon and muP

+ 4 pts Post-training

+ 4 pts Conditioning in NN

+ 4 pts Linear approximation perspective on Neural Nets

Question 3

ConvBasics **10** / 10 pts

3.1 **Conv3Params** **2** / 2 pts

✔ **+ 2 pts** Correct: $3 \times 3 \times 10 \times 20 + 20$

+ 1 pt Partial credit: correct number of weights $3 \times 3 \times 10 \times 20 = 1800$ but incorrect number of biases

+ 1 pt Partial credit: correct number of biases 20 but incorrect number of weights

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

3.2 **Conv3Outputs** **2** / 2 pts

✔ **+ 2 pts** Correct: $12 \times 12 \times 20$

+ 1 pt Partial: correct spatial dimension 12×12 but incorrect number of channels. Answers must have only 3 dimensions to get credit for this.

+ 1 pt Partial: correct number of channels 20 but incorrect spatial dimension. Answers must have only 3 dimensions to get credit for this.

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

3.3 **Conv1Params** **2** / 2 pts

✔ **+ 2 pts** Correct: $1 \times 1 \times 20 \times 20 + 20$

+ 1.5 pts Otherwise correct answer with carry-over error from previous part

+ 1 pt Partial credit: correct number of weights 1x1x20x20 = 400 but incorrect number of biases

+ 1 pt Partial credit: correct number of biases 20 but incorrect number of weights

+ 0 pts Incorrect

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

3.4 **Conv1Outputs** **2** / 2 pts

✔ **+ 2 pts** Correct: $6 \times 6 \times 20$

+ 1 pt Otherwise correct answer with carry-over error from a previous part

+ 1 pt Partial: correct spatial dimension 6×6 but incorrect number of channels. Answers must have only 3 dimensions to get credit for this.

+ 1 pt Partial: correct number of channels 20 but incorrect spatial dimension. Answers must have only 3 dimensions to get credit for this.

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

3.5 **LinParams** **2** / 2 pts

✔ **+ 2 pts** Correct: $720 \times 10 + 10$

+ 1 pt Partial credit: correct number of weights $720 \times 10 = 7200$ but incorrect number of biases

+ 1 pt Partial credit: correct number of biases 10 but incorrect number of weights

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

Question 4

DiffPool **12** / 12 pts

4.1 Softmax and Embeddings **6** / 6 pts

✔ **+ 6 pts** Correct:

$$X^{(1)} = \begin{bmatrix} 2 & 2.75 \\ 2 & 1.25 \end{bmatrix}$$

Softmax Computation Group

+ 5 pts

Partial Credit: Correctly computes $S_{\text{softmax}} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ \frac{1}{2} & \frac{1}{2} \\ \frac{1}{4} & \frac{3}{4} \end{bmatrix}$

+ 1.5 pts Partial credit: Incorrectly tries to compute the softmax across 4 entries (columns) which doesn't make sense in the DiffPool Context but correctly obtains $\begin{bmatrix} \frac{e^2}{e^2+e^3+e+1} & 0 \\ \frac{e^3}{e^2+e^3+e+1} & 0 \\ \frac{e}{e^2+e^3+e+1} & \frac{e}{e+3} \\ \frac{1}{e^2+e^3+e+1} & \frac{3}{e+3} \end{bmatrix}$

No partial credit will be given for incorrectly computing the softmax across columns

+ 2 pts Partial credit: attempts to correctly compute S_{softmax} across 2 entries (rows) but makes an error

---

+ 2 pts Partial credit: correctly computes the product $(S_{\text{softmax}})^T Z$ using the student's computed value of $(S_{\text{softmax}})^T$, even if the softmax computation is incorrect.

Answers which do not attempt to first normalize S by taking the softmax get no credit and are wrong.

+ 0 pts Incorrect, but not blank

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for cheating

4.2 LP Loss **3** / 3 pts

✔ **+ 3 pts** Correct: Align clusters to graph topology

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for cheating

4.3 **Entropy Loss** **3** / 3 pts

✔ **+ 3 pts** Correct: encourage hard assignments of nodes to clusters

+ 0 pts Incorrect but not blank

+ 0 pts Blank

Question 5

Muon

14 / 14 pts

5.1 **Updates** **6** / 6 pts

✔ **+ 6 pts** Entirely Correct.

GD = $-\begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix}$

SignGD = $-\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$

Muon = $-\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

Muon answer can be left unsimplified, e.g.

Muon = $-(1/2)\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$

It is OK if the student assumed averaging over the batch for GD, e.g.

GD = $-(1/3)\begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix}$

+ 4 pts Almost fully correct: All numbers are correct, but missing negative sign. Update should follow the *negative* gradient

---

GD Problem

+ 2 pts Entirely correct.

GD = $-\begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix}$

+ 2 pts Full credit, but assumed averaging over the batch.

GD = $-(1/3)\begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix}$

+ 2 pts Full credit, answer is unsimplified and contains a and b terms, but is otherwise correct. Must include negative sign.

GD = $-\begin{bmatrix} a+b & a-b \\ a-b & a+b \end{bmatrix}$

+ 1 pt Answer contains correct relative values, but incorrect by a constant factor.

GD = $-\begin{bmatrix} 3x & -x \\ -x & 3x \end{bmatrix}$

+ 1 pt Partial Credit: Incorrect answer due to arithmetic error or using wrong a and b values

+ 0 pts Incorrect -- just copied the given gradient definition without adding a negative sign.

GD = $\begin{bmatrix} a+b & a-b \\ a-b & a+b \end{bmatrix}$

+ 0 pts Entirely Incorrect

SignGD Problem

+ 2 pts Entirely correct.

SignGD = $-\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$

+ 2 pts Full credit, answer is unsimplified and contains a and b terms, but is otherwise correct. Must include negative sign.

GD = $-\begin{bmatrix} sign(a+b) & sign(a-b) \\ sign(a-b) & sign(a+b) \end{bmatrix}$

+ 1 pt Correct relative numbers, but incorrect by a constant factor.

SignGD = $-\begin{bmatrix} x & -x \\ -x & x \end{bmatrix}$

+ 1 pt Incorrect answer, but is a correct 'sign' transformation of calculated GD answer. Used if the student made an arithmetic error in calculating the GD answer, but still transformed it correctly for SignGD.

+ 0 pts Entirely incorrect.

Muon Problem

+ 2 pts Entirely Correct.

Muon = $-\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

+ 2 pts Unsimplified answer, but otherwise correct.

For example,
Muon = $-(1/2)\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$

+ 1 pt Relative numbers are correct, but scale is wrong by a constant factor.

Answer should take the form:
Muon = $-\begin{bmatrix} x & 0 \\ 0 & x \end{bmatrix}$

+ 1 pt Answer is otherwise correct, but student mistakenly transposed either U or V. Note that the SVD given was already in terms U and V^T, so an additional transpose was not necessary.

Work must be shown. A potential answer may be:

$$-(1/2)\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

Answer must include the (1/2) scaling.

+ 1 pt Partial Credit: Student writes a correct matrix multiplication down, but has an arithmetic error.

Work must be shown with:

$$UV^T = (1/2)\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

Answer must include the (1/2) scaling.

+ 0 pts Did not solve for the answer, i.e. there are a and b or 'SVD' terms in the answer

+ 0 pts Entirely incorrect

− 1 pt Modifier: Missing negative sign, update should follow the *negative* gradient.

+ 0 pts Blank

+ 0 pts Entirely Incorrect

+ 0 pts Flag for Professor

+ 0 pts Flag for potential chating

5.2 MoreCopiesChanges — 4 / 4 pts

✔ + 4 pts Just marked GD

+ 4 pts Alternate full credit: Didn't mark GD because they said averaging over batch for loss. And recognized that the others didn't change.

+ 0 pts Just marked SignGD

+ 0 pts Just marked Muon

+ 2 pts marked GD + SignGD

+ 0 pts marked SignGD + Muon

+ 2 pts marked GD + Muon

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for Suspected Cheating

5.3 ImbalanceChanges — 4 / 4 pts

✔ + 4 pts Fully Correct: Marked GD and SignGD

+ 2 pts Partial credit: Marked only GD.

SignGD also changes, as the sign flips on the off-diagonal terms.

+ 2 pts Partial Credit for work shown: Remarked that the Muon update is invariant to a/b

+ 0 pts Incorrect. This includes just marking SignGD since it is impossible for the SignGD update to change without the GD update also changing.

+ 0 pts Flag for Professor

+ 0 pts Blank

+ 0 pts Flag for potential cheating

Question 6

Quantization

14 / 14 pts

6.1 **Fill In Quantized Matrix** **9** / 9 pts

✔ **+ 9 pts** Fully Correct by visual inspection:

$$\begin{bmatrix} -1 & 3 & 2 \\ -4 & -1 & -1 \\ -1 & -4 & -1 \end{bmatrix}$$

Entry (1,1) is -1 since 0.25 is already quantized to -1 in entry (2,1).

Entry (1,2) is 3 since 2.0 = $r_{\max}$ and $q_{\max} = 3$.

Entry (3,1) is -1 since 0.25 < 0.4 < 0.5 and both 0.25 and 0.5 are quantized to -1.

+ 9 pts Fully Correct by computing the scale S and zero-point Z:

$$\begin{bmatrix} -1 & 3 & 2 \\ -4 & -1 & -1 \\ -1 & -4 & -1 \end{bmatrix}$$

$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} = \frac{3}{7}$

$Z = \text{round}(q_{\max} - \frac{r_{\max}}{S}) = \text{round}(q_{\min} - \frac{r_{\min}}{S}) = -\frac{5}{3}$

Then plugging entry (1,1) in the FP32 matrix in eq. (5) into eq. (4) to get -1.

Then plugging entry (1,2) in the FP32 matrix in eq. (5) into eq. (4) to get 3.

Then plugging entry (3,1) in the FP32 matrix in eq. (5) into eq. (4) to get -1.

Visual Inspection

+ 3 pts Noticed that entry (1,1) is -1 since 0.25 is already quantized to -1 in entry (2,1).

+ 3 pts Noticed that entry (1,2) is 3 since 2.0 = $r_{\max}$ and $q_{\max} = 3$.

+ 3 pts Noticed that entry (3,1) is -1 since 0.25 < 0.4 < 0.5 and both 0.25 and 0.5 are quantized to -1.

+ 1 pt Alternatively, used $q_{\max} = 2$ instead of $q_{\max} = 3$ for entry (1,2)

+ 1 pt Alternatively, used $q_{\max} = 4$ instead of $q_{\max} = 3$ for entry (1,2)

Computing scale and zero point

+ 3 pts Computed scale $S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}} = \frac{3}{7}$

+ 1 pt Wrote down system of equations to solve for the scale S but incorrectly computed S:

$r_{\max} = S(q_{\max} - Z) = 2 = S(3 - Z)$
and
$r_{\min} = S(q_{\min} - Z) = -1 = S(-4 - Z)$

+ 0.5 pts Only wrote down one of the equations to solve for the scale S:

$r_{\max} = S(q_{\max} - Z) = 2 = S(3 - Z)$
or
$r_{\min} = S(q_{\min} - Z) - 1 = S(-4 - -Z)$

+ 3 pts Computed zero point $Z = \text{round}(q_{\max} - \frac{r_{\max}}{S}) = \text{round}(q_{\min} - \frac{r_{\min}}{S}) = -\frac{5}{3}$

+ 1 pt Plugged entry (1,1) in the FP32 matrix in eq. (5) into eq. (4) and computed q correctly

+ 1 pt Plugged entry (1,2) in the FP32 matrix in eq. (5) into eq. (4) and computed q correctly

+ 1 pt Plugged entry (3,1) in the FP32 matrix in eq. (5) into eq. (4) and computed q correctly

---

+ 0 pts Blank

+ 0 pts Not blank but incorrect

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

6.2 Why Quantization Aware Training? 5 / 5 pts

✔ + 5 pts Fully Correct:
Nondifferentiability or zero derivatives

&

Learn to reduce quantization induced loss

+ 3 pts Partial Credit: (also marked the incorrect choice)

Nondifferentiability or zero derivatives

&

quantization saves memory during training

&

Learn to reduce quantization induced loss

+ 3 pts Nondifferentiability or zero derivatives

&

quantization saves memory during training

+ 1 pt Marked both incorrect choice and why we are doing QAT:

quantization saves memory during training

&

Learn to reduce quantization induced loss

+ 4 pts Just marked one correct choice: Nondifferentiability or zero derivatives

+ 4 pts Just marked one correct choice: Learn to reduce quantization induced loss

+ 0 pts quantization saves memory during training

+ 0 pts blank

+ 0 pts Flag for professor

+ 0 pts flag for suspected cheating

Question 7

DPO **24** / 24 pts

7.1 Rearrange Terms **6** / 6 pts

✔ **+ 6 pts** Completely correct: successfully manipulates equation (7) to obtain

$$r(x, y) = \beta \log \left(\frac{\mathcal{Z}(x)\, \pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} \right),$$

or

$$r(x, y) = \beta \log \left(\frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} \right) + \beta \log \mathcal{Z}(x),$$

or

$$r(x, y) = \beta \log \pi^*(y \mid x) - \beta \log \pi_{\text{ref}}(y \mid x) + \beta \log \mathcal{Z}(x),$$

or any algebraically equivalent form derived via standard logarithmic identities

+ 5 pts Partial credit: Almost correct but **ONLY** has some sign errors

+ 4 pts Partial credit: Had all the terms, but then dropped one (e.g., dropping $\beta \log \mathcal{Z}(x)$)

+ 0 pts Incorrect, but not blank. No credit for just starting the algebraic manipulation but not managing to correctly isolate $r(x, y)$ on one side.

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

7.2 Do we need $$Z$$? **4** / 4 pts

✔ **+ 4 pts** Completely correct: Selects that we do not need $Z(x)$ because it does not impact the difference $r(x, y_1) - r(x, y_2)$ because it contributes an identical additive offset

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

7.3 Where does the weight go? **3** / 3 pts

✔ **+ 3 pts** Completely correct: In such case, $\sigma(\hat{r}_\theta(x, y_\ell) - \hat{r}_\theta(x, y_w))$ approaches 1 for $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w))$

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

7.4 **Why DPO is better than SFT** **5** / 5 pts

✔ **+ 5 pts** Completely correct: **ONLY** selects the first **two** choices

- The first choice is correct because for every preference pair $(x, y_w y_\ell)$, SFT simply maximizes $\log(\pi_\theta(y_w \mid x))$ with uniform weight, regardless of whether the model already prefers y_w over y_ℓ
- The second choice is correct because SFT treats each winner y_w equally leading to the same amount of updates to each winner thus leaving the model prone to catastrophic forgetting during fine-tuning
- The third choice is incorrect because multiplying by numbers that are moderate sized like this is not enough to defend against overflow
- The fourth choice is incorrect because the issue of vanishing gradients is orthogonal to DPO. Its updates neither help nor hurt

+ 2.5 pts Partial credit: **ONLY** Selects the **first** choice

+ 2.5 pts Partial credit: **ONLY** Selects the **second** choice

+ 2.5 pts Partial credit: Selects **BOTH** of the **two** correct choices (the first and second choices) but also selects the incorrect **third** choice

+ 2 pts Partial credit: Selects **BOTH** of the **two** correct choices (the first and second choices) but selects the incorrect **fourth** choice

+ 0.5 pts Partial credit: Selects the **first** choice **AND** the **third** choice

+ 0.5 pts Partial credit: Selects the **first** choice **AND** the **fourth** choice

+ 0.5 pts Partial credit: Selects the **second** choice **AND** the **third** choice

+ 0.5 pts Partial credit: Selects the **second** choice **AND** the **fourth** choice

+ 0 pts Entirely incorrect, not blank

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

7.5 What to do about overfitting? 6 / 6 pts

✔ **+ 6 pts** Completely correct: **ONLY** selects the **three** correct choices (the first, third, and fifth choices)

- The first choice is correct because increasing β would constrain the policy from drifting too far from the reference policy thus lowering the KL divergence and keep it within a reasonable range of the reference policy space
- The second choice is incorrect because decreasing β even more would lead to an even higher KL divergence and thus a farther drift from the reference policy space
- The third choice is correct because it prevents the model from entering a (possibly) degenerate, high-KL regime even if other hyperparameters aren't perfectly tuned
- The fourth choice is incorrect because additional training would likely increase the KL divergence further and worsen the mode collapse the model has suffered
- The fifth choice is correct because LoRA fine-tuning requires far fewer trainable parameters than full fine-tuning, which helps prevent the model from experiencing a form of overfitting (to the reward estimation problem) that can manifest as catastrophic forgetting (of how to generate high-quality outputs).

+ 4 pts Partial credit: Selects the **first** and **third** choices but does **NOT** select any of the incorrect choices (the second and fourth choices)

+ 4 pts Partial credit: Selects the **first** and **fifth** choices but does **NOT** select any of the incorrect choices (the second and fourth choices)

+ 4 pts Partial credit: Selects the **third** and **fifth** choices but does **NOT** select any of the incorrect choices (the second and fourth choices)

+ 3 pts Partial credit: Selects **ALL** of the **three** correct choices (the first, third, and fifth choices) and selects the **second** choice

+ 3 pts Partial credit: Selects **ALL** of the **three** correct choices (the first, third, and fifth choices) and selects the **fourth** choice

+ 2.5 pts Partial credit: Selects the **first** and **third** choices and selects the **fourth** choice

+ 2.5 pts Partial credit: Selects the **first** and **fifth** choices and selects the **fourth** choice

+ 2.5 pts Partial credit: Selects the **third** and **fifth** choices and selects the **fourth** choice

+ 2.5 pts Partial credit: Selects the **first** and **third** choices and selects the **second** choice

+ 2.5 pts Partial credit: Selects the **first** and **fifth** choices and selects the **second** choice

+ 2.5 pts Partial credit: Selects the **third** and **fifth** choices and selects the **second** choice

+ 2 pts Partial credit: Selects the **first** choice but does **NOT** select any of the incorrect choices (the second and fourth choices)

+ 2 pts Partial credit: Selects the **third** choice but does **NOT** select any of the incorrect choices (the second and fourth choices)

+ 2 pts Partial credit: Selects the **fifth** choice but does **NOT** select any of the incorrect choices (the second and fourth choices)

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

Question 8

Using Parameters to Impact Learning Rate

10 / 15 pts

+ 15 pts Entirely Correct.

$$y = 10(Wx + b)$$
$$W \sim N(0, \frac{1}{d_{\text{in}} * 100})$$
$$b \sim N(0, 10^{-6})$$

Solution to y

+ 7 pts Entirely Correct.

$$y = 10(Wx + b)$$

+ 7 pts Full credit, expresses new y in terms of a previous y

y = 10y

+ 4 pts Partial credit: Missing factor of 10 multiplication on b.

$$y = 10Wx + b$$

✔ **+ 4 pts** Partial Credit: Multiplied by an incorrect power of 10, e.g. $100, \sqrt{10}$.

+ 4 pts Partial Credit: Multiplied W and b by different values, but one of them is correct.

+ 3 pts Partial credit: Divided by 10 or $\sqrt{10}$ rather than multiplying.

+ 2 pts Partial Credit: Multiplied by an unrelated factor, e.g. d_{in} or η.

+ 0 pts Incorrect; did not modify original equation.

+ 0 pts Entirely Incorrect

Solution to W

+ 4 pts Entirely Correct.

$$W \sim N(0, \frac{1}{d_{\text{in}} * 100})$$

✔ **+ 3 pts** Partial credit: Divided variance by 10 instead of 100. Remember that the second argument is variance, not standard deviation.

$$W \sim N(0, \frac{1}{d_{\text{in}} * 10})$$

+ 2 pts Partial credit: Divided variance by $\sqrt{(10)}$ rather than 100.

$$W \sim N(0, \frac{1}{d_{\text{in}} * \sqrt{10}})$$

+ 2 pts Partial credit: Multiplied variance by 100 rather than dividing.

$$W \sim N(0, \frac{100}{d_{\text{in}}})$$

+ 1 pt Partial credit: Multiplied or divided variance by an otherwise incorrect value, e.g. $sqrt(10), sqrt(d_{\text{in}})$.

Includes answers of $W \sim N(0, \frac{10}{d_{\text{in}}})$

+ 0 pts Incorrect; did not modify original equation.

+ 0 pts Incorrect; attempts to square root the dimension term.

+ 0 pts Incorrect; attempts to add a term rather than multiply.

+ 0 pts Entirely Incorrect

Solution to b

+ 4 pts Entirely correct.

$b \sim N(0, 10^{-6})$

✔ **+ 3 pts** Partial credit: Divided variance by 10 rather than 100.

$b \sim N(0, 10^{-5})$

+ 2 pts Partial credit: Multiplied variance by 100 rather than dividing.

$b \sim N(0, 10^{-2})$

+ 1 pt Partial credit: Multiplied or divided variance by an otherwise incorrect value, e.g. multiplied by 10, divided by $\sqrt{(10)}$, or any answer involving d_{in}.

Includes answers of $b \sim N(0, 10^{-3})$

+ 0 pts Incorrect; did not modify original equation.

+ 0 pts Incorrect, attempts to add a term rather than multiply.

+ 0 pts Entirely incorrect.

+ 0 pts Blank

+ 0 pts Entirely Incorrect

+ 0 pts Flag for professor

+ 0 pts Suspected of potential cheating

Question 9

Debugging

12 / 12 pts

9.1 **The case of the missing causal mask** **6** / 6 pts

✔ **+ 6 pts** Correct: Identified that causal masking is missing in this implementation.

Correct Bug Group

+ 6 pts Model needs to not be paying attention to future tokens.

+ 0 pts AlsoABug: Create new entries for every distinct thing that people say that is actually also a bug in the code.

NotABugGroup

+ 0 pts Stated normalization was not included or incorrect: the code already includes normalization via division by sqrt(d_model) before the softmax, which is the standard scaling used in attention.

+ 0 pts Stated that the issue is missing KV cache updates: incorrect. KV caching is an inference-time optimization used for efficient autoregressive decoding and is not required for correctness of causal attention itself.

+ 0 pts Claimed that the softmax is applied over the wrong dimension: incorrect. The attention scores tensor has shape (batch_size, seq_len, seq_len), and applying softmax over dim = -1 correctly normalizes over the key dimension for each query position independently within each batch element.

+ 0 pts Claimed nonlinearity was not present: incorrect. Softmax is a non linear transformation and feed forward layers provide non linearities in later stages.

+ 0 pts Stated that the model shape is wrong: incorrect. All tensor shapes in the code are consistent

+ 0 pts Positional encoding is not strictly necessary in attention mechanisms. For example, old-style additive positional encoding doesn't do positional encodings inside Causal attention itself. NoPE is also an important approach that does no positional encoding inside the attention mechanism. RoPE does exist within the attention mechanism, but nothing tells us that the problem requires RoPE.

+ 0 pts Claimed that the code is incorrect because it does not implement multi-head attention or cross-attention: incorrect. Multi-head attention and cross-attention are architectural choices used in full Transformer models but are not required for a correct implementation of causal self-attention.i

+ 0 pts Claimed that the code is incorrect because a layer or parameter is not defined: incorrect. The snippet shows only the forward function, not the module constructor. Layers and parameters are typically defined in the init method and referenced in forward.

+ 0 pts Claimed that there is a structural or ordering error in the attention computation: incorrect. The attention operations shown follow the standard scaled dot-product attention formulation with valid tensor shapes and correct operation order. No such structural error is present.

+ 0 pts Blank

+ 0 pts Incorrect but not blank: use this only for cases where the answer is so wrong or off-topic that it cannot be categorized in the NotABug group

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

9.2 **The case of the missing KL Divergence** **6** / 6 pts

✔ **+ 6 pts** Correct: Identified that the KL divergence penalty is missing in the implementation. This could be as simple as saying that there is no auxiliary loss since only the end-to-end reconstruction loss is there.

Also A Bug Group

+ 3 pts Identified a mismatch between training and sampling latent distributions but did not explain that this occurs because the KL divergence term is missing from the loss.

+ 3 pts Says loss is missing a term but gives no indication of understanding that this is an auxiliary loss or a regularization type term.

+ 0 pts AlsoABug: Create new entries for every distinct thing that people say that is actually also a bug in the code.

NotABugGroup

+ 0 pts Claimed that the decoder is used incorrectly and that the model does not properly sample from the encoder distribution or that the noise is being generated incorrectly: incorrect. The code already samples z using the reparameterization trick

+ 0 pts log_std is wrong, incorrect. Here it is being used to enforce positive values. The specific choice here has a diagonal matrix for the covariance. This is a different approach that using the square-root of the matrix, but also works.

+ 0 pts Wrong type of loss. This is mostly a design choice, mse is used here because x is assumed to be data such as images for which cross entropy would not work well.

+ 0 pts Claims that we shouldn't be sampling noise in the loss and that this should instead be done in the encoder itself. Again, this is a design choice of where to do this. It is not a bug

+ 0 pts Claims the torch.randn function is incorrect. This returns a sample of a Gaussian with zero mean and unit variance. So using it is correct in this context.

+ 0 pts Claims a shape error. But z_mean and z_log_std can have the same shape. There is no shape error.

+ 0 pts Claims we need to add noise to the output of the decoder. This is false. The whole point is to add noise to the input of the decoder so it is robust to noise.

+ 0 pts Claims normalization is missing.

+ 0 pts Claims we're missing a command to prevent gradient accumulation across batches. But that happens near where we use the optimizer, not in the loss calculation.

+ 0 pts Blank

+ 0 pts Incorrect but not blank: use this only for cases where the answer is so wrong or off-topic that it cannot be categorized in the NotABug group. Claiming that there is no role for random noise is in this category. Similarly, confusing a VAE decoder with the expected-noise-prediction step in denoising diffusion models, falls into this category.

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

Question 10

Update Ranks **24** / 24 pts

10.1 **Just a linear layer with a single example** **6** / 6 pts

✔ **+ 6 pts** Correct: $\Delta W = -gx^T$

+ 5 pts Almost correct: sign error (just writes gx^T)

+ 4 pts Wrong shape, but conceptually close: writes the transpose of the update $-xg^T$

+ 3.5 pts Partial credit: Otherwise correct update scaled by a factor other than the given $\eta = 1$

+ 3 pts Double mistake partial credit: writes the transpose of the update *and* includes a sign error: xg^T

+ 3 pts Double mistake partial credit: writes the transpose of the update $-xg^T$ *and* scales the update by a factor other than $\eta = 1$

+ 2.5 pts Partial credit: Writes the SignSGD update $-\text{sgn}(gx^T)$

+ 1.5 pts Partial credit: writes an inner product instead of an outer product (writes x^Tg, g^Tx, $x \cdot g$, or $g \cdot x$. These answers are all equivalent).

+ 1 pt Partial credit: writes an inner product, but scales the update by a factor other than $\eta = 1$

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

10.2 Linear layer with a batch — 3 / 3 pts

✔ **+ 3 pts** Correct: $\Delta W = -GX^T$

+ 3 pts Correct alternate answer: $\Delta W = -\frac{1}{B}GX^T$

This is for standard averaging of gradients across the batch.

+ 2 pts Partial credit: sign error (just writes GX^T)

+ 2 pts Partial credit: alternate answer with sign error (just writes $\frac{1}{B}GX^T$)

+ 1.5 pts Partial credit: writes the SignSGD update $-\text{sgn}(GX^T)$

+ 1 pt Partial credit: sign error and transposed gradient (i.e. XG^T)

+ 1 pt Partial credit: sign error and forgets final transpose gradient (i.e. XG)

+ 1 pt Partial credit: sign error and concatenates column-wise outer products instead of adding

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

10.3 Rank of batch case — 3 / 3 pts

✔ **+ 3 pts** Correct: $\text{rank}(\Delta W) = B$. Answers must explicitly mention B for full credit.

+ 1 pt Partial credit: B times a (positive) constant

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

10.4 Output channels — 2 / 2 pts

✔ **+ 2 pts** Correct: C_{out}

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

10.5 Shape of input — 4 / 4 pts

✔ **+ 4 pts** Correct: input should have shape $(K \cdot K \cdot C_{in}) \times (W \cdot H)$

+ 3 pts Partial credit: swaps row and column dimensions and writes $(W \cdot H) \times (K \cdot K \cdot C_{in})$

+ 2 pts Partial credit: Correctly notes that the first dimension should be $(K \cdot K \cdot C_{in})$ but the second dimension is incorrect

+ 2 pts Partial credit: Correctly notes that the second dimension should be $(W \cdot H)$ but the first dimension is incorrect

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

10.6 Convolutional Update — 3 / 3 pts

✔ **+ 3 pts** Correct: $\Delta W = -GX^\top$

+ 2 pts Partial credit: sign error (just writes GX^T)

+ 2 pts Partial Credit: forgot the transpose and wrote $-GX$ instead of $-GX^\top$.

+ 1 pt Partial credit: sign error and transposed gradient (i.e. XG^T)

+ 1 pt Partial Credit: sign error and forgot the transpose: wrote GX instead of $-GX^\top$.

+ 0 pts Incorrect

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

10.7 **Rank Bound** **1.5** / 1.5 pts

✔ **+ 1.5 pts** Correct: $\min(C_{\text{out}}, K \cdot K \cdot C_{\text{in}}, W \cdot H)$, and all 3 terms in the minimum are explicitly written out.

+ 1 pt Partial credit: one term in the minimum expression is incorrect or omitted (submissions where 2 or more terms are incorrect or not explicitly stated get only 0.5 point)

+ 0.5 pts Partial credit: notices the rank bound should be the minimum of 2 or more terms

+ 0.75 pts Partial credit: states a loose upper bound of C_{out}, $W \cdot H$, or $K \cdot K \cdot C_{in}$ with no minimum over multiple terms

+ 0 pts Incorrect, but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for cheating

10.8 **Explanation of rank bound** **1.5** / 1.5 pts

✔ **+ 1.5 pts** Fully Correct: Any answer that acknowledges that a single image is actually a collection of many patches from the perspective of a conv-net.

Note that these patches are over the spatial dimension of the image and are *not* the same as having multiple filters or input channels.

+ 0 pts Incorrect but not blank

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

Question 11

muP **22** / 24 pts

11.1 Basic Case **8** / 8 pts

✔ **+ 8 pts** Fully Correct $O(\frac{1}{d_{\text{in}}})$

+ 4 pts Incorrect, $O(\frac{1}{\sqrt{d_{\text{in}}}})$ but depends on the right thing in a reasonable way.

+ 2 pts Incorrect, $O(\sqrt{d_{\text{in}}})$ but depends on the right thing but in an unreasonable way --- it grows instead of shrinks.

+ 2 pts Partial Credit: Solved for $||\Delta\vec{y}||_{\text{RMS}}$ in terms of ΔW.

+ 0 pts Incorrect: Depends on d_{out}

+ 0 pts Incorrect: Constant

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

+ 0 pts Blank

11.2 Embedding Case **8** / 8 pts

✔ **+ 8 pts** Correct: $O(1)$

+ 2 pts Incorrect, but identified that generally d_{in} is the right dimension to scale by.

$$O(1/\sqrt{d_{\text{in}}})$$
$$O(1/d_{\text{in}})$$
$$O(\sqrt{d_{\text{in}}})$$
$$O(d_{\text{in}})$$

+ 2 pts Partial credit for work shown: Found $||\vec{y}||_{\text{RMS}}$ in terms of ΔE.

+ 3 pts Partial credit for work shown: Derived that the RMS norm of the one-hot embedding is $O(1/\sqrt{d_{\text{in}}})$

+ 0 pts Incorrect

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

11.3 **Ratio with same learning-rate** **2** / 4 pts

+ 4 pts Entirely Correct: $1 : d_{\text{in}}$

✔ **+ 2 pts** Incorrect, uses a square instead $1 : d_{\text{in}}^2$

+ 1 pt Incorrect, swapped ratio order.

$$d_{\text{in}} : 1$$

+ 0.5 pts Incorrect, swapped ratio and uses a squared term.

$$d_{\text{in}}^2 : 1$$

+ 1.5 pts Partial Credit: Identified the RMS contributions to y of the ΔW term. (Optionally, in terms of learning rate, but OK if they assumed learning rate = 1).

$$\|\Delta \vec{y}_{\text{from weight update}}\|_{\text{RMS}}^2 = \|(\Delta W)\vec{x}\|_{\text{RMS}}^2 = O(d_{\text{in}})$$

+ 1.5 pts Partial Credit: Identified the RMS contributions to y of the Δb term. (Optionally, in terms of learning rate, but OK if they assumed learning rate = 1).

$$\|\Delta \vec{y}_{\text{from biases}}\|_{\text{RMS}}^2 = \|\Delta \vec{b}\|_{\text{RMS}}^2 = O(1)$$

+ 0 pts Incorrect

+ 0 pts Blank

+ 0 pts Invalid, selected multiple answers

+ 0 pts Flag for Professor

+ 0 pts Suspected of potential cheating

11.4 **Ratio with muP rate** **4** / 4 pts

✔ **+ 4 pts** Correct: $1 : 1$

+ 1.5 pts Partial Credit: Identified the RMS contributions to y of the ΔW term. (Optionally, in terms of learning rate, but OK if they assumed learning rate = 1)

i.e. If ΔW is the muP-transformed update, then
$\|\Delta \vec{y}_{\text{From the weights}}\|^2_{\text{RMS}} = \|(\Delta W)\vec{x}\|^2_{\text{RMS}} = O(1)$

+ 1.5 pts Partial Credit: Identified the RMS contributions to y of the Δb term. (Optionally, in terms of learning rate, but OK if they assumed learning rate = 1).

$\|\Delta \vec{y}_{\text{from biases}}\|^2_{\text{RMS}} = \|\Delta \vec{b}\|^2_{\text{RMS}} = O(1)$

+ 1 pt Incorrect, but identifies that RMS contributions are generally a function of d_{in}.

$$1 : d_{\text{in}}$$
$$1 : d_{\text{in}}^2$$
$$d_{\text{in}} : 1$$
$$d_{\text{in}}^2 : 1$$

+ 0 pts Incorrect

+ 0 pts Blank

+ 0 pts Invalid, selected multiple answers

+ 0 pts Flag for Professor

+ 0 pts Suspected of potential cheating

Question 12

Comparison Table **19.5** / 20 pts

12.1 **Transformer Training Compute Order** **3** / 3 pts

✔ **+ 3 pts** Completely Correct

Got runtime of $O(T^2 d)$

+ 2.5 pts Partial Credit:

Correct Q, K, V computation and realized QK^T dominates runtime with $O(T^2)$ dependence, but used $O(d^2)$ for computation of each score when computing dot products only takes $O(d)$ time

Ended with $O(T^2 d^2)$

+ 2 pts Partial Credit:

Realized attention scores $QK^\top$ dominates runtime but got wrong runtime.

Ended with $O(T^2)$

+ 1.5 pts Partial Credit:

Correct Q, K, V computation but did not account for attention matrix computation for QK^T

Ended with answer like $O(Td^2)$

+ 1 pt Partial Credit:

Wrong dependence on T and d in computing the runtimes of Q, K, V matrices or attention scores.

Ended with answer like $O(Td)$ or $O(T)$

+ 0.5 pts Partial Credit:

Considered some computation of embeddings and scores, but did not show attempt at computing Q, K, V matrcies or attention scores.

Ended with answer like $O(d)$ or $O(d^3)$

+ 0 pts Blank

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

12.2 SSM Training Compute Order — 2.5 / 3 pts

+ 3 pts Completely Correct

Got runtime of $O(Td \log T + Td^2)$

+ 3 pts Correct but arrived at different runtime (just $O(Td^2)$) by considering chunking the sequence (explanation must mention how $\log T$ factor was eliminated.

+ 2.5 pts Partial Credit:

Correctly computed runtime for states and each convolutional kernel using FFT, but made slight error (wrong dimension in FFT or forgot d different kernels)

Ended with answer like $O(T \log T + Td^2)$.

✔ **+ 2.5 pts** Partial credit:

Got runtime of convolution using FFT correctly, but did not include time to get states (it is unclear which of Td^2 or $Td \log T$ is bigger)

Ended with $O(Td \log T)$

+ 2 pts Partial Credit:

Correctly identified convolution operation of d kernels but got incorrect runtime.

Ended with answer like $O(Td^2 \log T)$ or $O(T \log T)$

+ 1.5 pts Partial Credit:

Did not identify convolution operation and only noted runtime of computing states.

Ended with answer like $O(Td^2)$

+ 1 pt Partial credit:

Considered some computation but did not show computing states nor convolution.

Ended with answer like $O(dN)$ or $O(d^2)$ or $O(TN)$

+ 0 pts Blank

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Flag for suspected cheating

+ 0 pts Flag for professor

12.3 **RNN Inference Compute Order** **3** / 3 pts

✔ **+ 3 pts** Completely Correct

Got runtime of $O(Td^2)$

+ 2.5 pts Partial Credit:

Identified need to update hidden state, but incorrectly computed runtime (updates to hidden parameter
require O(d^2) computation).

Ended with $O(Td)$

+ 1.5 pts Partial Credit:

Correct computation of per-token update requiring $O(d^2)$ computation, but incorrect dependence on sequence length.

Ended with answer like $O(T^2d^2)$

+ 1 pt Partial credit:

Considered some computation of hidden state, but got incorrect dependence on T and d.

Ended with answer like $O(d^3)$ or $O(d^2)$

+ 0 pts Blank

+ 0 pts Flag for suspected cheating

+ 0 pts Flag for professor

+ 0 pts Entirely incorrect, but not blank

12.4 **Transformer Inference Compute Order** **3** / 3 pts

✔ **+ 3 pts** Completely Correct

Got runtime of $O(T^2 d)$

+ 3 pts Correct in absence of KV-cache (justification is all correct assuming no KV-cache)

Ended with $O(T^2 d^2)$ and correctly noted it was due to recomputing K, V matrices multiple times.

+ 2.5 pts Partial Credit:

Almost correct computation but used $O(d^2)$ computation for each entry in attention score instead of $O(d)$ for dot product.

Ended with $O(T^2 d^2)$

+ 2 pts Partial Credit:

Realized runtime was dominated computation of attention scores, but got incorrect runtime.

Ended with $O(T^2)$

+ 1.5 pts Partial Credit:

Correct computation of Q, K, V matrices (with or without KV-cache), but did not realized runtime dominated by attention scores, getting wrong dependence on sequence length at end (each token depends on all prior tokens, leading to O(T^2) dependence on runtime).

Ended with answer like $O(T d^2)$

+ 1 pt Partial Credit:

Considered some computation of attention scores, but got incorrect dependence on T and/or d.

Ended with answer like $O(T)$ or $O(d^2)$

+ 0.5 pts Partial Credit:

Considered some computation of embeddings but did not demonstrate attempt at computing attention scores or Q, K, V matrices.

Ended with answer like $O(d)$ or $O(d^3)$

+ 0 pts Blank

+ 0 pts Flag for suspected cheating

+ 0 pts Flag for professor

+ 0 pts Entirely incorrect, but not blank

12.5 **Transformer Training Explanation** **2** / 2 pts

✔ **+ 2 pts** Correct

Explained that compute is dominated by self-attention that takes $O(T^2 d)$ time.

+ 1.75 pts Partial Credit:

Explanation has correct components but wrong runtime.

+ 1 pt Partial Credit:

Explanation talks about computing embeddings but not attention scores, leading to incorrect dependence on T

+ 1 pt Partial Credit:

Explanation does not account for computation due to matrix products (for Q, K V matrices and projections), leading to incorrect dependent on d (i.e. assumed constant time computation of each attention score).

+ 0.5 pts Partial Credit:

Runtime is wrong and explanation does not hint towards computing Q, K, V, nor attention scores $QK^\top$

+ 0 pts Entirely incorrect, but not blank. For example, explanation is just a runtime (and runtime is wrong)

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

12.6 **SSM Training Explanation** **2** / 2 pts

✔ **+ 2 pts** Correct

Explained that runtime is due to computing states in $O(Td^2)$ time and convolution over states in $(Td \log T)$ time.

+ 1.75 pts Partial Credit:

Explanation correctly identifies time to compute states, and convolution, but gets wrong runtimes (i.e. incorrect application of FFT)

+ 1 pt Partial Credit:

Explanation mentions computing input-output states, but does not mention convolution with kernel of length T using states.

+ 0 pts Entirely incorrect, but not blank. For example, explanation is just runtime (and runtime is wrong)

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

+ 2 pts Correct if student put $O(Td^2)$ runtime and correctly noted that $\log T$ dependence can be surpressed by chunking the sequence to maximally exploit GPU parallelism opportunities by computing convolutions over chunks, and then propagating state between chunks.

we do not know if d >> log T so we want to include both in runtime.

12.7 **RNN Inference Explanation** **2** / 2 pts

✔ **+ 2 pts** Correct

Explanation mentions generating T tokens taking $O(d^2)$ time for each hidden state update.

+ 1.75 pts Partial Credit:

Explanation has correct components (hidden state update, generating token-by-token, etc.) but use wrong runtimes.

+ 1 pt Partial Credit:

Incorrect analysis of sequence length dimension, leading to incorrect dependence on T in final answer.

+ 1 pt Partial Credit:

Mentioned T tokens but not cost of update of hidden state per-token, leading to incorrect dependence on d in final answer.

+ 0.5 pts Partial Credit:

Runtime is wrong and explanation does not reference hidden state update, nor autoregressive generation.

+ 0 pts Entirely incorrect, but not blank. For example, explanation is just runtime (and runtime is wrong)

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

12.8 **Transformer Inference Explanation** **2** / 2 pts

✔ **+ 2 pts** Correct

Explanation mentions $O(Td)$ time to compute attention scores for each token, and summing over T tokens.

+ 1.75 pts Partial Credit:

Explanation has correct components (compute attention score for each token and sum) but wrong runtime for computing attention scores for each token

+ 1 pt Partial Credit:

Explanation mentions computing Q, K, V, but not attention scores, arriving at wrong dependence on T.

+ 1 pt Partial Credit:

Explanation talks about computing attention scores, but has incorrect analysis of computation. This includes mistakenly saying those can be obtained without computation due to KV-cache.

+ 1 pt Partial Credit:

Wrong runtime and explanation only refers to training cost and not what is different (i-th token takes $O(id)$ cost to generate then summing leads to $O(T^2d)$)

+ 0.5 pts Partial Credit:

Wrong runtime and explanation does not mention computing Q, K, V nor attention scores $QK^\top$

+ 0 pts Entirely incorrect, but not blank. For example, explanation is just runtime (and runtime is wrong)

+ 0 pts Blank

+ 0 pts Flag for professor

+ 0 pts Flag for suspected cheating

Question 13

PreNorms **20** / 20 pts

13.1 **Expression from block diagram** **3** / 3 pts

✔ **+ 3 pts** Fully Correct: $X_1 = X_0 + W_2\text{ReLU}(W_1\text{LayerNorm}(X_0))$

+ 3 pts Alternatively Fully Correct (Using row-vectors): $X_1 = X_0 + \text{ReLU}(\text{LayerNorm}(X_0)W_1)W_2$

Must be clear that they are using rows.

+ 1.5 pts Partial Credit: The X_0 term is correct

+ 1.5 pts Partial Credit: the $W_2\text{ReLU}(W_1\text{LayerNorm}(X_0))$ term is correct

+ 0 pts Blank

+ 0 pts Incorrect, but not blank.

+ 0 pts Flag for Professor

+ 0 pts Flag for possible cheating

13.2 **Jacobian of Residual Block** **5** / 5 pts

✔ **+ 5 pts** Fully Correct (column-wise):

$$\frac{\partial X_1}{\partial X_0} = I + W_2 D W_1 J$$

The residual connection preserves gradient norms better because the Jacobian contains the identity I. The presence of I ensures that even if the function term $W_2 D W_1 J$ is close to the zero matrix, the over- all Jacobian is not vanishing. When we multiply together many such terms, the product of the identity matrices will always give at least one identity term — this represents the "gradient superhighway" that residual connections enable.

+ 5 pts Fully Correct (row-wise):

$$\frac{\partial X_1}{\partial X_0} = I + J W_1 D W_2$$

The residual connection preserves gradient norms better because the Jacobian contains the identity I. The presence of I ensures that even if the function term $W_2 D W_1 J$ is close to the zero matrix, the over- all Jacobian is not vanishing. When we multiply together many such terms, the product of the identity matrices will always give at least one identity term — this represents the "gradient superhighway" that residual connections enable.

+ 5 pts Fully correct:

Correctly computed Jacobian of an erroneous expression for X_1 in question 13 (a).

The residual connection preserves gradient norms better because the Jacobian contains the identity I. The presence of I ensures that even if the function term $W_2 D W_1 J$ is close to the zero matrix, the over- all Jacobian is not vanishing. When we multiply together many such terms, the product of the identity matrices will always give at least one identity term — this represents the "gradient superhighway" that residual connections enable.

+ 2 pts Partial credit for residual connection: Got the Identity term.

+ 2 pts Partial credit (column-wise): Got the term $W_2 D W_1 J$.

+ 2 pts Partial credit (row-wise): Got the term $J W_1 D W_2$.

+ 1 pt Partial Credit: Explains that the residual connection preserves gradient norms better because the Jacobian contains the identity I. The presence of I ensures that even if the function term $W_2 D W_1 J$ is close to the zero matrix, the over- all Jacobian is not vanishing. When we multiply together many such terms, the product of the identity matrices will always give at least one identity term — this represents the "gradient superhighway" that residual connections enable.

– 1 pt Added unnecessary transposes

+ 0 pts Blank

+ 0 pts Incorrect, but not blank.

+ 0 pts Flag for Professor

+ 0 pts Flag for possible cheating

13.3 **RMSNorm Layer Jacobian** **12** / 12 pts

✔ **+ 12 pts** Completely Correct

$$J = \frac{\sqrt{d}}{\|\vec{X}\|_2}(I - \frac{\vec{X}\vec{X}^\top}{\|\vec{X}\|_2^2})$$

Or any form that is identical to this as your final answer. If you make errors after reaching a form that is identical, that does not count as completely correct.

+ 0 pts Blank

Partial Credit: Vector-Calculus Approach

+ 1 pt Set up equation for the Jacobian

$$J = \frac{\partial Z}{\partial X} = \frac{\partial}{\partial X}(\sqrt{d}\frac{\vec{X}}{\|\vec{X}\|_2})$$

+ 2 pts Decomposed the Jacobian using the product rule

$$J = \sqrt{d}(\frac{1}{\|\vec{X}\|_2}I + \vec{X}\frac{\partial}{\partial X}(\frac{1}{\|\vec{X}\|_2}))$$

+ 2 pts Attempted to compute $\frac{\partial}{\partial X}(\frac{1}{\|\vec{X}\|_2})$

+ 4 pts Correctly computed

$$\frac{\partial}{\partial X}(\frac{1}{\|\vec{X}\|_2}) = -\frac{1}{\|\vec{X}\|_2^2}\vec{X}^\top$$

+ 3 pts Correctly simplified the expression for the Jacobian to get $J = \frac{\sqrt{d}}{\|\vec{X}\|_2}(I - \frac{\vec{X}\vec{X}^\top}{\|\vec{X}\|_2^2})$ or equivalent form.

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

Partial Credit: Locally Linear Interpretation Approach

+ 2 pts Analyzed $R(X + \Delta X)$ when ΔX is in same direction as X.

+ 2 pts Correctly showed that if ΔX is aligned with X, it must be that $J\Delta X = 0$

+ 1 pt Analyzed $R(X + \Delta X)$ when ΔX is orthonormal to X

+ 3 pts If ΔX is orthogonal, correctly compute the updated RMSNorm as

$$||X + \Delta X||_2 \approx ||X||_2 + \frac{||\Delta X||_2^2}{2||X||_2}$$

which means that $||X + \Delta X||_2$ is unchanged up to first-order by ΔX.

+ 2 pts Correctly showed that if ΔX is orthogonal, then

$$J\Delta X \approx \frac{\sqrt{d}}{||X||_2}\Delta X$$

+ 2 pts Combine both results to conclude that if J is a linear map, then it must take the form

$J = \frac{\sqrt{d}}{||X||_2}\left(I - UU^\top\right)$ for $U = \frac{X}{||X||_2}$

– 1.5 pts Error during computation of Jacobian (i.e incorrect exponent, tranpose, etc.)

– 1 pt Minor error during computation of Jacobian (i.e. flipped sign)

Question 14

Discrete Diffusion **28** / 28 pts

14.1 **Things approach pure noise** **4** / 4 pts

✔ **+ 4 pts** Completely correct: $\lim_{t\to\inf} \mathrm{P}(\text{the pixel has not been replaced until T}) = \lim_{t\to\inf}(1-2\delta)^T = 0$, since $0 < 1-2\delta < 1$

Partial credit choices when the argument is wrong/flawed

+ 4 pts This exists as a base to take points off from using a modifier.

Almost completely correct: $\lim_{t\to\inf} \mathrm{P}(\text{the pixel has not been replaced until T}) = \lim_{t\to\inf}(1-2\delta)^T = 0$, since $0 < 1-2\delta < 1$

+ 2 pts Partial Credit: Derive the not flipping probability at time step t leveraging iid condition. $\mathrm{P}(\text{the given pixel has not been replaced until t }) = (1-2\delta)^t$

+ 1 pt Partial Credit: Derive the complementary probability correctly. $\mathrm{P}(\text{the given pixel is not replaced}) = 1-2\delta$

+ 1 pt Partial credit: Somewhat correct answer with incorrect derivation.

− 1 pt Modifier: Used δ instead of 2δ

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

14.2 Sampling for Training — 6 / 6 pts

✔ **+ 6 pts** Completely correct:
With probability $1-(1-2\delta)^t$ we will overwrite the bit with a second random (50-50 chance) probability, otherwise we leave it alone.
Also accepted to do this in one go by replacing the bit with probability $\frac{1-(1-2\delta)^t}{2}$

Explanation: At each time step, there is a 2δ probability of us randomly reassigning the bit with 50-50 probability. For these bits, because multiple random reassignments throughout the forward process still result in a 50-50 chance of being 1 or -1 at the end of forward diffusion, we can wait to do this reassignment at the end. If we haven't decided to eventually do a bit flip after t time steps (happens with $(1-2\delta)^t$ probability), then we leave it alone, otherwise (in the $1-(1-2\delta)^t$ probability case where we have decided to reassign at least once) we randomly reassign the bit with 50-50 probability

+ 3 pts Partial Credit:
Identified the "if else" structure where
if we've left the bit alone at each time step, the bit stays the same,
else (meaning we have randomly reassigned it at some point) we need to randomly reassign the bit

+ 2 pts Partial Credit:
Identified that after t time steps, the probability that we leave the coin alone (ie: not overwriting it with noise) is $(1-2\delta)^t$ OR the probability that we change the bit is $1-(1-2\delta)^t$

+ 1 pt Partial Credit:
Identified that in the case where we flip the bit, we do it with 50-50 probability of being heads or tails

+ 3 pts Partial Credit:
Used the binomial distribution instead, without mentioning how to efficiently sample from it that would be as efficient as a method involving only two coin flips

+ 0 pts Blank

+ 0 pts Entirely incorrect, but not blank

Answers that try to simply flip one coin per pixel fall into this category.

Answers that actually go through all the t time steps also fall here.

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

+ 1 pt Partial Credit: some correct work

14.3 Nonlinearity for Probabilities — 2 / 2 pts

✔ **+ 2 pts** Completely correct: Marked Sigmoid only

+ 1 pt Partial Credit: marked Sigmoid and tanh

+ 0.5 pts Partial Credit: marked tanh only.

+ 0 pts Blank

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

14.4 **Conditional Probability Target** **10** / 10 pts

✔ **+ 10 pts** Completely correct: (using the helper result)

$$p(Y_{t-1}[j]=1|y[j]=1,x[j]=1)=\frac{(1-\delta)\rho}{(1-\rho)\delta+(1-\delta)\rho}$$

$$p(Y_{t-1}[j]=1|y[j]=-1,x[j]=1)=\frac{\delta\rho}{\rho\delta+(1-\delta)(1-\rho)}$$

where δ is the flip probability given in the original question, and

$\rho=\frac{1}{2}(1+(1-2\delta)^{t-1})$

+ 10 pts Completely correct: (done directly without the helper)

$$p(Y_{t-1}[j]=1|y[j]=1,x[j]=1)=\frac{(1-\delta)\rho}{q}$$

$$p(Y_{t-1}[j]=1|y[j]=-1,x[j]=1)=\frac{\delta\rho}{1-q}$$

where δ is the flip probability given in the original question, and

$\rho=\frac{1}{2}(1+(1-2\delta)^{t-1})$ is $p(Y_{t-1}[j]=1|x[j]=1)$, and

$q=\frac{1}{2}(1+(1-2\delta)^{t})$ is $p(Y_{t}[j]=1|x[j]=1)$. Note that $1-q=\frac{1}{2}(1-(1-2\delta)^{t})$ and the answer might appear in this form.

Note also that some answers in this form will have cancelled the $\frac{1}{2}$ on top and bottom and might appear as:
$p(Y_{t-1}[j]=1|y[j]=1,x[j]=1)=\frac{(1-\delta)(1+(1-2\delta)^{t-1})}{1+(1-2\delta)^{t}}$

and

$$p(Y_{t-1}[j]=1|y[j]=-1,x[j]=1)=\frac{\delta(1+(1-2\delta)^{t-1})}{1-(1-2\delta)^{t}}$$

Bayes Rule using the hint (if Bayes Rule eqns are correct, the student gets all subparts in this box)

+ 2 pts Identify (maybe implicitly) that A in the hint formula is $Y_{t-1}[j]|Y_0[j]=1$. In other words, A is the random variable corresponding to $Y_{t-1}[j]$ with the relevant universe (sigma-field) being conditioned on $Y_0[j]=x[j]=1$.

The particular *event* we are interested in is $A=1$.

+ 2 pts Identify (maybe implicitly) that B in the hint formula is $Y_t[j]|Y_0[j]=+1.$. In other words, B is the random variable corresponding to $Y_t[j]$ with the relevant universe (sigma-field) being conditioned on $Y_0[j]=x[j]=1$.

We are interested in both events $B=+1$ and $B=-1$

+ 1 pt Answer plugs values into the relevant hint formulas: (3) and (5)

Bayes rule without using the hint (if Bayes Rule eqns are correct, the student gets all subparts in this box) Can't get these points if the previous group points were being given

+ 2 pts Identify that $p(Y_t[j]=1|Y_{t-1}[j]=-1,Y_0[j]=1)=p(Y_t[j]=1|Y_{t-1}[j]=-1)=\delta$ (equivalently, that $p(Y_t[j]=1|Y_{t-1}[j]=1,Y_0[j]=1)=p(Y_t[j]=1|Y_{t-1}[j]=1)=1-\delta$)

+ 2 pts Set up the correct Bayes rule formula $p(A|B) = p(A)p(B|A)/p(B)$, for $p(A) = p(Y_{t-1}[j] = 1|Y_0[j] = 1)$ and $p(B) = p(Y_t[j] = 1|Y_0[j] = -1)$

These are all the points obtainable by someone who does not make the connection to δ.

+ 1 pt Correctly plug values into the Bayes rule chosen

Finding ρ (i.e. finding $p(Y_{t-1}[j] = 1|Y_0[j] = 1)$)

+ 2 pts Formula is P(pixel never replaced) + P(pixel replaced) * ½

+ 2 pts P(pixel never replaced) is $(1 - 2\delta)^{t-1}$

+ 1 pt P(pixel replaced) is 1 - p(pixel never replaced)

+ 2 pts Partial Credit: Reasonable attempt which doesn't result in the right answer (if you get this rubric item, you cannot get any other partial credit

– 1 pt Partial Credit: Modifier: algebra error

– 2 pts Partial Credit: Modifier: correctly computes one of $p(Y_{t-1}[j] = 1|Y_0 = 1, x[j] = 1)$ or $p(Y_{t-1}[j] = 1|Y_0 = -1, x[j] = 1)$, but the other is missing/wrong

+ 0 pts Blank

+ 0 pts Entirely incorrect (or very very incomplete), but not blank

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

14.5 Interpret Resulting KL Loss — 6 / 6 pts

✔ **+ 6 pts** Completely correct: Correctly states that the loss is penalized most strongly the probability that the neural net says that something is very unlikely when the proxy target distribution is very strongly concentrated there. Correctly finds that the loss at $t = 1$ looks like

$$\sum_{j=1}^{m} -\log Q_{1,j}(x_i[j]|Y_1)$$

and recognizes that this is a sum of binary cross-entropy losses for denoising the individual pixels $x_i[j]$ of the slightly noisy image Y_1, or alternatively, looks at the real scores before the sigmoid nonlinearity and recognizes the connection to logistic loss.

Partial Credit group for what the loss penalizes most strongly

+ 3 pts Identifies that the KL divergence penalizes the network having very low probability at a location where the target distribution has a high density.

+ 1.5 pts Makes a more generic statement, such as "the loss is penalized most at locations where the prediction differs significantly from the target distribution" or "Q is near zero and P is nonzero" (not specifically noting that P is large)

Partial Credit group for the loss when $t = 1$

+ 3 pts Correctly notices that the loss on Q looks like binary cross-entropy loss for recovering the original image. Also get credit if they look back to the real scores coming out of the neural net that preceded the sigmoid and say that this looks like logistic loss on those pixel-level scores.

$$\sum_{j=1}^{m} -\log Q_{1,j}(x_i[j]|Y_1)$$

Extreme precision is not required --- if it is clear that they understand that this is what is going on, they will get the points.

+ 2 pts Not only did they notice that the target probability distribution is deterministic, but they recognize the independence of the randomness across the probabilities for different binary digits and turn the \log of a probability into a sum.

+ 1 pt Noticed that the target probability distribution is deterministic for $t = 1$ and hence the expectation in the definition of the KL divergence has only a single term.

+ 0 pts Entirely incorrect, but not blank

+ 0 pts Blank

+ 0 pts Flag for Professor

+ 0 pts Flag for suspected cheating

Question 15

Doodle Page

0 / 0 pts

+ **0 pts** Blank

✔ + **0 pts** Gratitude

✔ + **0 pts** Drawing

+ **0 pts** Smiley

+ **0 pts** Work for another problem

+ **0 pts** Hearts

+ **0 pts** Another language...

✔ + **0 pts** Lament

+ **0 pts** EECS

✔ + **0 pts** Flag for professor

+ **0 pts** Comment

+ **0 pts** Tracking exam progress.

+ **0 pts** Humor

+ **0 pts** Cheating report.

+ **0 pts** Missing Doodle Page

+ **0 pts** Celebrating last final as an undergrad

+ **0 pts** Comment on ID check, but not a cheating type report.

EECS 182 Deep Neural Networks
Fall 2025 Anant Sahai and Gireeja Ranade

Final Exam

Exam location: Evans 10

PRINT your student ID: 3041994738

PRINT AND SIGN your name: Wen, Jie Jie (Menger) Wen
(last) (first) (signature)

PRINT your discussion section: Wednesday 4-5pm
Row Number (front row is 1): most behind Seat Number (left most is 1): most right

Name and SID of the person to your left: _____

Name and SID of the person to your right: _____

Name and SID of the person in front of you: Siddharth Shashi 3038374835

Name and SID of the person behind you: Fangzhou Zhao 3039228454

Section 0: Pre-exam questions (5 points)

1. Honor Code: Please copy the following statement in the space provided below and sign your name below. (1 point or $-\infty$)

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules and do this exam on my own.

As a member of the UC Berkeley community,
I act with honesty, integrity, and respect for others.
I will follow the rules and do this exam on my own. Jie Wen

2. What's your favorite deep learning topic? (4 points)

Transformers : Attention is all I need (in studying)

Do not turn this page until the proctor tells you to do so. You can work on Section 0 above before time starts.

Final Exam, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

3. CNN Basics (10 pts)

We are going to describe a convolutional neural net using the following pieces:

- CONV(K, F, P, S) denotes a convolutional layer with F different filters, each of size $K \times K \times C$, where C is the depth (i.e. number of channels) of the activations from the previous layer. Padding is P, and stride is S.
- POOL(2) denotes a 2×2 max-pooling layer with stride 2 (pad 0)
- FLAT just turns whatever shape input tensor into a one-dimensional array with the same values in it.
- LIN(D) denotes a single fully-connected linear layer with D output activations. The number of input activations implicitly comes from the previous layer.

Note: All CONV and LIN layers have biases as well as weights. *Do not forget the biases when counting parameters.*

Now, we are going to use this network to do inference on a single input. **Fill in the missing entries in this table of the size of the number of parameters at each layer and the shape of the output activations at each layer. You can/should write your answer as a computation** (e.g. $128 \times 128 \times 3$) in the style of the already filled-in entries of the table. You don't have to evaluate it, but that can help check your work at the end.

Layer	Number of Parameters	Dimension of Activations in output of layer
Input	0	$26 \times 26 \times 1$
CONV(5, 10, 1, 1)	$5 \times 5 \times 1 \times 10 + 10$	$24 \times 24 \times 10$
POOL(2)	0	$12 \times 12 \times 10$
CONV(3, 20, 1, 1)	$3 \times 3 \times 10 \times 20 + 20$	$12 \times 12 \times 20$
CONV(1, 20, 0, 2)	$1 \times 1 \times 20 \times 20 + 20$	$6 \times 6 \times 20$
FLAT	0	720
LIN(10)	$720 \times 10 + 10$	10

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

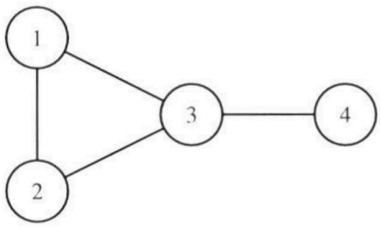
[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

Final Exam, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

4. DiffPool (12 pts)

Consider the following graph with 4 nodes:



Suppose each node has 2-dimensional features given by feature matrix X where row i of X is the feature vector for node i:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & 0 \end{bmatrix}$$

You will perform **one DiffPool layer**:

$$4 \text{ nodes} \rightarrow 2 \text{ clusters.}$$

Suppose we additionally have GNNs which give the following (**unnormalized**) scores (in S) for each node and embeddings (in Z):

$$S = \begin{bmatrix} 2 & -\infty \\ 3 & -\infty \\ 1 & 1 \\ 0 & \ln 3 \end{bmatrix}, \qquad Z = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 1 \\ 2 & 1 \end{bmatrix},$$

(a) (6pts) **Compute the node embeddings at the output of the diffpool:**

$$X^{(1)} = (S_{\text{softmax}})^{\top} Z.$$

Remember that $S_{softmax}$ takes the cluster scores for each node and turns them into a probability via the softmax operation (which takes the exponential of scores and then normalizes all those positive numbers into a probability).

Each row: $[2, +\infty] \rightarrow (1, 0)$; $[3, -\infty] \rightarrow (1, 0)$; $[1, 1] \rightarrow (\frac{1}{2}, \frac{1}{2})$; $[0, \ln 3] \rightarrow (\frac{1}{4}, \frac{3}{4})$

$$\therefore S_{softmax} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ \frac{1}{2} & \frac{1}{2} \\ \frac{1}{4} & \frac{3}{4} \end{bmatrix}$$

Cluster 1: $\sum_i S_{i1} z_i = 1 z_1 + \frac{1}{2} z_3 + \frac{1}{4} z_4 = (2, \frac{11}{4})$

Cluster 2: $\sum_i S_{i2} z_i = \frac{1}{2} z_3 + \frac{3}{4} z_4 = (2, \frac{5}{4})$

$$\therefore X^{(1)} = \begin{bmatrix} 2 & 2.75 \\ 2 & 1.25 \end{bmatrix}$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

(b) (6pts) Recall that to make training more effective, DiffPool adds auxiliary losses.

For each of the following auxiliary losses, **select the primary design purpose of using the auxiliary loss.**

i. LP loss: $\|A - S_{\text{softmax}} S_{\text{softmax}}^{\top}\|_F^2$ where A is the adjacency matrix of the graph.

(●) Align clusters to graph topology ○ Encourage hard assignments of nodes to clusters ○ Other

ii. Average entropy minimization loss[1]: $-\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{K} S_{\text{softmax}}[i,j]\log S_{\text{softmax}}[i,j]$

○ Align clusters to graph topology (●) Encourage hard assignments of nodes to clusters ○ Other

[1]Note: this loss is non-negative despite the seeming negative sign. Because this is a cheat-sheet-free exam, we tell you: the entropy of a probability distribution is always non-negative, is maximized by the uniform distribution, and is minimized by putting all the probability on a single possibility.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304994738

5. Muon and Other Optimizers (14 pts)

In this question, we will focus on an idealized scenario where we are training a single linear layer with weight matrix W, and our batch consists entirely of copies of two datapoints whose individual gradients $\nabla_W L$ are:

$$\textbf{Gradient wrt. W for datapoint A}: \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \textbf{Gradient wrt. W for datapoint B}: \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad (1)$$

Assume that the training batch consists entirely of a copies of datapoint A and b copies of datapoint B. The overall full-batch gradient is now[2]:

$$\textbf{Gradient}: \begin{bmatrix} a+b & a-b \\ a-b & a+b \end{bmatrix} \quad \text{which has SVD:} \quad \left(\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\right)\begin{bmatrix} 2a & 0 \\ 0 & 2b \end{bmatrix}\left(\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}\right). \quad (2)$$

(a) (6pts) Assume $a = 1, b = 2$. **What is the resulting update ΔW when the gradient is used by gradient descent, SignGD, and Muon respectively?** Assume a learning rate of 1 and no momentum. Assume an idealized[3] Muon that takes the gradient and sets all its singular values to 1 instead of iterating through a matrix polynomial.

GD: $\Delta W = -G = \begin{bmatrix} -3 & 1 \\ 1 & -3 \end{bmatrix}$

SignGD: $\begin{bmatrix} -1 & +1 \\ +1 & -1 \end{bmatrix}$

Muon: $\begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$

[2] For simplicity, we are assuming our loss sums over points in the batch instead of averaging over them.

[3] You can further ignore the matrix-shape-based terms in calculating the Muon update that allow the learning rate to be understood in terms of an RMS-Norm-to-RMS-Norm induced norm ball. This footnote exists because of the no clarifications, no questions policy. If you didn't have a question, you can ignore this footnote.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

(b) (4pts) You increase your batch by inserting more copies of A and B at the same $1:2$ ratio. **Which of the three updates change from part (a)?**

☑ GD ☐ SignGD ☐ Muon

(c) (4pts) You now have a differently imbalanced training batch where $a = 10, b = 1$. **Which of the three updates above change from part (a)?**

☑ GD ☑ SignGD ☐ Muon

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304994738

6. Quantization Aware Training (14 pts)

Recall from homework that linear quantization is defined by the goal of

$$r \approx S(q - Z) \tag{3}$$

where r is a floating point real number, q is a n-bit integer, Z is a n-bit integer, and S is a floating point real number. Z is the quantization zero point and S is quantization scaling factor. Both Z and S are quantization parameters that can be kept at higher precision because their cost will be amortized over lots of entries in a weight matrix being quantized together.

The actual quantization is done using rounding to the nearest integer:

$$q = \text{round}(\frac{r}{S} + Z). \tag{4}$$

Consider a 3×3 weight matrix whose values are represented in standard FP32 (float) format as

$$\begin{bmatrix} 0.25 & 2.0 & 1.5 \\ -1.0 & 0.5 & 0.25 \\ 0.4 & -1.0 & 0.5 \end{bmatrix}. \tag{5}$$

Now, you would like to use linear quantization to represent this weight matrix using 3 bits for q, so $q_{\min} = -4$ and $q_{\max} = 3$, and the eight possible values for q are: -4,-3,-2,-1,0,1,2,3. Let $r_{\min} = -1.0$ and $r_{\max} = 2.0$ be the smallest and largest values in the weight matrix. In order to use the full dynamic range of q, we should correspond $q_{\min}$ to $r_{\min}$ and $q_{\max}$ to $r_{\max}$.

(a) (9pts) What does the weight matrix (5) look like *after* applying linear quantization using the best computed S and Z values? Carefully look at the partially filled entries and **complete the empty entries in the Q matrix below:**

$$Q = \begin{bmatrix} \boxed{-1} & \boxed{3} & 2 \\ -4 & -1 & -1 \\ \boxed{-1} & -4 & -1 \end{bmatrix} \tag{6}$$

(Hint: While you can solve for the blanks using explicit computation of the S and Z and applying (4), you can also get the values by inspection and thinking about what must happen in quantization.)

$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}} = \frac{3}{7}$ $\quad Z = \text{round}\left(q_{min} - \frac{r_{min}}{S}\right) = \text{round}\left(-4 + \frac{7}{3}\right) = 2$

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304199473 8

(b) (5pts) Quantization-aware training quantizes the full-precision model weights w to w_q during each forward pass of training. It uses these quantized weights for computing activations. For backpropagation, we apply the "straight-through" estimator to compute the backpropped gradient through the quantization step to the underlying full-precision weights. This straight-through estimator treats the effect of quantization as though it were random external noise added to the full-precision weights. **Why do we do this?** (Select all that apply)

- ☑ Quantization is either nondifferentiable (when viewed as a function into the integers) or has zero derivatives almost everywhere (when viewed as a function from $\mathbb{R} \to \mathbb{R}$) and would prevent meaningful updates if we actually differentiated it.
- ☐ Because quantization saves memory during training when storing the momentum terms in the optimizer.
- ☑ Because we want the final model to be used after quantization for inference, it is useful to allow training to learn how to reduce the loss due to quantization.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

7. Your Language Model is Secretly a Reward Model (24 pts)

In homework, you worked through the foundations of Direct Preference Optimization (DPO). If we optimize our policy $\pi_\theta(y|x)$ to maximize our reward model $r(x, y)$ (that tells us quantitatively how good y is as a response to x) while trying to stay close to a reference policy $\pi_{\text{ref}}(y|x)$ in KL Divergence:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(y|x)} \left[r(x, y)\right] - \beta\mathcal{D}_{\text{KL}}[\pi_\theta(y|x)\|\pi_{\text{ref}}(y|x)]$$

Then, the optimal solution takes the form:

$$\pi^*(y|x) = \frac{1}{\mathcal{Z}(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x, y)\right) \tag{7}$$

where $\mathcal{Z}(x) = \sum_y \pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x, y)\right)$ normalizes the probability and β is the hyperparameter that determines how much we want to stay close to the reference distribution π_{ref}. Here, x is the prompt/question and y is the response from the model.

(a) (6pts) **Manipulate (7) to express $r(x, y)$ in terms of $\pi^*(y|x)$, $\pi_{\text{ref}}(y|x)$, β, and $\mathcal{Z}(x)$.**

For (7), divided by π_{ref} on both sides and take log:

$$\frac{\pi^*(y|x)\mathcal{Z}(x)}{\pi_{ref}(y|x)} = \exp\left(\frac{r(x,y)}{\beta}\right) \Rightarrow r(x,y) = \beta \log \frac{\pi^*(y|x)\mathcal{Z}(x)}{\pi_{ref}(y|x)}$$

Also can be written: $r(x,y) = \beta\left(\log\pi^*(y|x) - \log\pi_{ref}(y|x) + \log\mathcal{Z}(x)\right)$

(b) (4pts) Looking at your expression in the previous part, **is knowledge of $\mathcal{Z}(x)$ required to understand if the implicit reward prefers response y_1 to y_2 in response to prompt/question x?**

- ○ Yes, we need $\mathcal{Z}(x)$
- ☑ No, we do not need $\mathcal{Z}(x)$

$r(x,y_1) - r(x,y_2)$: $\log\mathcal{Z}(x)$ will be cancelled out!

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

(c) (8pts) Now, consider the DPO gradient you derived in homework for training a network using a dataset consisting of triplets: two responses for each prompt x. The response that humans prefer is y_w and the response that humans don't prefer is y_ℓ.

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\beta \mathbb{E}_{(x,y_w,y_\ell)\sim\mathcal{D}} \left[\underbrace{\sigma\left(\hat{r}_\theta(x, y_\ell) - \hat{r}_\theta(x, y_w)\right)}_{\text{weight}} \left[\underbrace{\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_\ell|x)}_{\text{update direction}} \right] \right]$$

where σ is the sigmoid function $\sigma(z) = 1/(1+e^{-z})$, $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the implicit reward, and the distribution $\mathcal{D}$ just represents what SGD is doing.

(i) Consider a case where the model is extremely confident that the dispreferred response y_ℓ is actually better than the preferred response y_w (i.e., $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$). **What happens to the weight term in the gradient, i.e. $\sigma(\hat{r}_\theta(x, y_\ell) - \hat{r}_\theta(x, y_w))$?** Select one response.

- ✓ The gradient weight becomes substantial, approaching 1 if $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$.
- ○ The gradient weight approaches zero if $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$.
- ○ The gradient weight approaches $\frac{1}{2}$ if $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$.

(ii) Why is the adaptive-weighting in DPO desirable compared to the standard supervised fine-tuning (SFT) approach that simply maximizes $\mathbb{E}_{(x,y_w)\sim\mathcal{D}} \log\left(\pi_\theta(y_w|x)\right)$? (**Mark all that apply**)

- ✓ Our preference training data just tells us which of two responses were better. It doesn't tell us if both were somewhat bad or both were good but one was better. And yet, standard SFT would treat all the preferred responses as though they were gold-standard perfect responses and completely ignore the information present in knowing which alternatives are less preferred.
- ✓ Because standard SFT would apply the same amount of update towards winners even when they were already winning, it is more prone to triggering catastrophic forgetting in the model during a fine-tune, especially as we fine-tune for multiple epochs. The DPO updates are comparatively more minimal.
- ☐ Attenuating the strength of the gradients in this DPO way helps prevent numerical overflow as compared to standard SFT.
- ☐ Doing the DPO update better prevents vanishing gradients.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304199438

(d) (6pts) Imagine you are full fine-tuning a 7B instruction-following model using DPO with β selected from the low end of the typical range and reference policy $\pi_{\text{ref}} = \pi_{\text{SFT}}$. After several epochs of training on preference pairs, you observe:

- Training loss: decreased significantly from initial values
- KL divergence $\mathcal{D}_{\text{KL}}[\pi_\theta(y|x)\|\pi_{\text{ref}}(y|x)]$: has grown substantially larger than typically recommended
- Generated responses: contain repetitive phrases and degenerate patterns. They have gotten worse than what π_{SFT} generated.
- Held-out preference accuracy: very high (model correctly ranks test preferences better than π_{SFT} did)
- Human evaluation on new prompts: responses are often incoherent or low-quality

Below are potential strategies for addressing the training issues identified above. **Select ALL strategies that would likely be helpful.**

- ☑ Increase β substantially (e.g., by a factor of 3-5x) to strengthen the KL penalty and stay closer to the reference policy.
- ☐ Decrease β to allow the model more freedom to optimize the preference objective.
- ☑ Implement early stopping.
- ☐ Continue training for additional epochs to allow the model to find better solutions.
- ☑ Use DPO with a LoRA fine-tune that trains fewer learnable parameters instead of a full fine-tune.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

8. Using Redundancy in Parameterization (15 pts)

Assume you are given the following dense layer:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \tag{8}$$

with the shape of the weights W being $d_{\text{out}} \times d_{\text{in}}$ and the biases $\mathbf{b}$ being d_{out}-dimensional. There is an iid random Gaussian initialization of:

$$W[i,j] \sim N(0, \frac{1}{d_{\text{in}}}) \qquad b[i] \sim N(0, 10^{-4}) \tag{9}$$

for i, j ranging over the shape of the weight matrix and bias vector. Here, the first argument to N is the mean and the second argument is the variance.

You are dealing with a particularly annoying codebase in which you cannot adjust the Adam optimizer being used or change any of the optimizer parameters, including the learning rate, momentum terms, etc.[4] Feel free to think about the SignGD optimizer as standing-in for Adam in this problem.

However, you are able to change the network definition in (8) as well as the initialization described by (9). **How can you adjust these definitions in** (8) **and** (9) **so that the input-output behavior training trajectory matches what would happen if you had used a 10x higher learning rate for Adam in the original network** (8) **and initialization** (9)**?**

$\mathbf{y} =$ $\sqrt{10}\,(W\mathbf{x} + \mathbf{b})$

$W[i,j] \sim$ $N(0, \frac{1}{10\, d_{in}})$

$b[i] \sim$ $N(0, \frac{10^{-4}}{10})$

[4] To avoid technicalities, assume that the ϵ parameter in Adam that prevents division by zero is just $\epsilon = 0$. You can ignore this footnote if you weren't worried about that part — this exists only because this is a zero-questions, zero-clarifications exam.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304994738

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

9. Debugging Generative Models (12 pts)

You are training your first generative model, and you're very excited as your training and validation losses have both dropped to nearly zero. But when you try to generate some samples, only garbage comes out! What is wrong or missing in each of the following implementations, and/or how would you fix it? **Answer briefly. You don't need to give the exact fix in code, just tell us what you need to do.** If you say X is missing, we will understand that the implied fix is to include X.

(a) (6pts)

```
class CausalAttention(nn.Module):
    def forward(self, x):
        Q = self.W_q(x)  # (batch_size, seq_len, d_model)
        K = self.W_k(x)    # (batch_size, seq_len, d_model)
        V = self.W_v(x)  # (batch_size, seq_len, d_model)
        scores = torch.matmul(Q, K.transpose(-2, -1))
        scores = scores / math.sqrt(self.d_model)
        attention_weights = F.softmax(scores, dim=-1)
        attn_output = torch.matmul(attention_weights, V)
        output = self.W_o(attn_output)
        return output
```

Above, we have what should be a causal attention block used as part of a standard Transformer language model recipe for a GPT-style model pretrained using batches consisting of teacher-forced entire sequences and next-token prediction loss. To sample from the model, we would generate tokens autoregressively. **What is wrong with the above implementation for training?**

Lack of causal mask: now every position can look at the future tokens
Under teacher forcing, the training loss will be very low,
but it will crush when generating.

Fix: Before softmax, add a triangle $-\infty$ mask to scores or use 'torch.tril' etc.
to guarantee that only attend up to ≤ now position

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

(b) (6pts)

```
class VariationalAutoEncoder(nn.Module):
    def get_loss(self, x):
        # Encode x.
        z_mean, z_log_std = self.encoder(x)

        # Add noise to z.
        noise = torch.randn(z_mean.shape)
        z = z_mean + torch.exp(z_log_std) * noise

        # Decode z back to x, with mean-squared reconstruction loss.
        x_pred = self.decoder(z)
        loss = F.mse_loss(x, x_pred)
        return loss
```

Above, we have what should be the entire loss function for training a variational auto-encoder. To sample from this model, we would sample $z \sim N(0, 1)$, and decode with our learned decoder. **What is wrong with the above implementation for training?**

Lack of KL regularization (ELBO $KL[q_\phi(z|x) \| N(0,1)]$)

The codes only reconstructed MSE and it will turn into normal autoencoders

Sampling $z \sim N(0,1)$ and decoding will become rubbish

Fix: loss = recon_loss + KL

if encoder outputs μ, $\log\sigma$ (or $\log var$),

KL can be written as the normal enclosed form

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

10. Rank of Updates (24 pts)

Throughout this problem, assume a learning rate of $\eta = 1$ and simple SGD with no momentum is being used to do weight updates.

(a) (6pts) Let $\mathbf{y} = W\mathbf{x}$ represent a fully connected layer where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is the input to the layer, and $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$ is the output of the layer.

This layer is being trained with a minibatch consisting of a single input, and backprop from the final loss brings a nonzero upstream gradient $\mathbf{g} = \nabla_{\mathbf{y}} L$ where L is the final loss in our deep network. Note that $\mathbf{g}$ has the same shape as $\mathbf{y}$ since it is a gradient.

Recall that: $\frac{\partial L}{\partial W[i,j]} = x[j]\frac{\partial L}{\partial y[i]}$ for $\mathbf{y} = W\mathbf{x}$.

What is the SGD update of the weight matrix, ie ΔW? (Your answer should be in terms of $\mathbf{x}, \mathbf{g}$.)

$$\Delta W = \underline{\quad -gx^T(\eta=1) \quad}$$

$$\text{rank}(\Delta W) = 1$$

(b) (6pts) Now, assume we process a batch of B training samples at a time. That is, our input is now a matrix $X = [\mathbf{x}_1\ \mathbf{x}_2\ \cdots\ \mathbf{x}_B]$ with shape $d_{\text{in}} \times B$ and our upstream gradient (from backprop) is now a corresponding matrix G with shape $d_{\text{out}} \times B$ where the i-th column is $\nabla_{\mathbf{y}_i} L_i$ the gradient of the loss with respect to the layer's output with input $\mathbf{x}_i$. Assume that $B < \min(d_{\text{in}}, d_{\text{out}})$. **What is the new update of the weight matrix ΔW?**

$$\Delta W = \underline{\quad -GX^T \quad}$$

What is the rank of this update? Assume the inputs in the batch are linearly independent and the upstream gradient also has full rank.

$$\text{rank}(\Delta W) = \underline{\quad B \quad}$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

(c) (6pts) Now, consider a convolutional layer with C_{in} input channels and C_{out} output channels, where each convolutional filter has spatial size $K \times K$. We will use same padding and a stride of 1 so that if the layer's input has size $W \times H \times C_{\text{in}}$, then the layer's output has size $W \times H \times C_{\text{out}}$.

Recall that such a convolutional layer can be represented as a linear layer, wherein all the filters are flattened together into a single matrix F and the input image is reshaped/recast into a matrix X so that each column of X corresponds to all the C_{in} input channels in a flattened $K \times K$ patch of the input image. **Fill in the following blanks:**

i. The reshaped filter F has matrix shape $C_{\text{out}} \times \underline{K \cdot K \cdot C_{\text{in}}}$.

ii. The output $Y = FX$ of this operation has shape $\underline{\quad C_{out} \quad} \times \underline{W \cdot H}$.
(Hint: The output can be reshaped into an image of shape $W \times H \times C_{out}$. Make sure your answer is consistent!)

iii. To be able to properly represent this convolutional layer, the input X should have shape:
$\underline{(K \cdot K \cdot C_{in})} \times \underline{\quad (W \cdot H) \quad}$.

(d) (6pts) Building on the previous part where $Y = FX$, assume we are updating the weights of this convolutional layer after seeing a single training image. **What is the update of the (flattened) weight matrix ΔF in terms of X, which is the reshaped input image, and upstream gradient $G = \nabla_Y L$?** (The upstream gradient has also been shaped to match the operation output Y above.)

$$\Delta F = \underline{\quad -\eta G X^T \quad}$$

Even though we have only a single training image, how big can the rank of the update matrix ΔF be? Your answer should be in terms of $K, C_{\text{in}}, C_{\text{out}}, W, H$.

$$\text{rank}(\Delta F) \leq \underline{\quad \min\{C_{out}, K^2 C_{in}, WH\} \quad}$$

Briefly explain why this rank can be strictly bigger than 1 while in part (a), the rank is 1 when there's a single training example in the batch.

Single image after im2col corresponds to WH "position items"
Updating is the sum of multiple position contributions
not single outer product anymore

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

11. muP (24 pts)

The maximal update parameterization (muP) provides a helpful scaling law for learning rates, such that training dynamics remain similar as we adjust the size of our neural network. Throughout this problem, assume we are utilizing an off-the-shelf SignGD optimizer (standing in as an idealization of Adam) with a minibatch size of 1.

(a) (8pts) Consider a bias-free dense linear layer defined as $\mathbf{y} = W\mathbf{x}$, where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. Assume $\mathbf{x}$ has RMS Norm 1. **How should we scale the learning rate for W so that the RMS-norm change in $\mathbf{y}$ remains constant regardless of the shape of W?**
(Hint: Remember part (a) of Problem 10: "Rank of Updates" to help.)

Select one:

○ $O(1)$ i.e. constant	○ $O(d_{\text{in}}d_{\text{out}})$	○ $O(\frac{d_{\text{in}}}{d_{\text{out}}})$
○ $O(\frac{1}{\sqrt{d_{\text{in}}}})$	○ $O(\frac{1}{\sqrt{d_{\text{out}}}})$	○ $O(\sqrt{\frac{2}{d_{\text{in}}+d_{\text{out}}}})$
✓ $O(\frac{1}{d_{\text{in}}})$	○ $O(\frac{1}{d_{\text{out}}})$	○ $O(\frac{2}{d_{\text{in}}+d_{\text{out}}})$
○ $O(\sqrt{d_{\text{in}}})$	○ $O(\sqrt{d_{\text{out}}})$	○ $O(\sqrt{\frac{d_{\text{in}}+d_{\text{out}}}{2}})$
○ $O(d_{\text{in}})$	○ $O(d_{\text{out}})$	○ $O(\frac{d_{\text{in}}+d_{\text{out}}}{2})$

$\Delta y = \Delta W x$

To each output dimension k:

$\Delta y_k = \sum_{i=1}^{d_{in}} (\pm\eta) x$

If the sign can be considered as independent and schocastic:

$\text{Var}(\Delta y_k) \approx \eta^2 \sum_i x_i^2 = \eta^2 \|x\|_2^2$

$\|x\|_{RMS} = 1 \Rightarrow \|x\|_2^2 = d_{in}$

$\therefore \text{std}(\Delta y_k) \approx \eta\sqrt{d_{in}}$ $\therefore \eta \sim O(\frac{1}{\sqrt{d_{in}}})$

(b) (8pts) Let us now consider embedding layers of a language model, parameterized by a matrix $E \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. Recall that performing lookup on an embedding layer can be understood as multiplying by a *one-hot* vector of dimension d_{in}. **How should we scale the learning rate for the embedding layer in terms of its shape to ensure that the RMS-norm change in the output of the embedding layer remains constant regardless of d_{in} and d_{out}.**

Select one:

✓ $O(1)$ i.e. constant	○ $O(d_{\text{in}}d_{\text{out}})$	○ $O(\frac{d_{\text{in}}}{d_{\text{out}}})$
○ $O(\frac{1}{\sqrt{d_{\text{in}}}})$	○ $O(\frac{1}{\sqrt{d_{\text{out}}}})$	○ $O(\sqrt{\frac{2}{d_{\text{in}}+d_{\text{out}}}})$
○ $O(\frac{1}{d_{\text{in}}})$	○ $O(\frac{1}{d_{\text{out}}})$	○ $O(\frac{2}{d_{\text{in}}+d_{\text{out}}})$
○ $O(\sqrt{d_{\text{in}}})$	○ $O(\sqrt{d_{\text{out}}})$	○ $O(\sqrt{\frac{d_{\text{in}}+d_{\text{out}}}{2}})$
○ $O(d_{\text{in}})$	○ $O(d_{\text{out}})$	○ $O(\frac{d_{\text{in}}+d_{\text{out}}}{2})$

One-hot will only use one column.
Gradients will act on this column as well.
SignSGD will update each entry in this column by $\pm\eta$
$\therefore \Delta y = \Delta E_x = \Delta E_{:i}$
Each element will change by η
$\therefore \|\Delta y\|_{RMS} = \eta$
do not depend on d_{in}/d_{out}

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304199738

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304199473 8

(c) (8pts) We will now consider learnable bias vectors. Consider a dense layer defined as $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$. Again, assume that $\mathbf{x}$ has RMSNorm 1.

i. Suppose we did *not* use muP scaling – i.e. we use the same learning rate for all parameters, regardless of shape – **how much does an update to the bias vector $\Delta\mathbf{b}$ contribute to the corresponding expected RMS-norm-squared of the change in $\mathbf{y}$, in comparison to the update to the weight matrix ΔW?** Express your answer as a rough proportion contribution-from-bias : contribution-from-dense-weight-matrix.
(Hint: For simplicity, think of the input $\mathbf{x}$ as being a vector entirely of 1s and the gradient $\nabla_{\mathbf{y}} L$ coming from upstream as also a vector consisting entirely of 1s.)

Select one:

○ $1 : 1$	i.e. same contribution	○ $1 : 2$	i.e. Bias contributes half as much
○ $1 : d_{\text{in}}$		○ $1 : d_{\text{out}}$	
☑ $1 : d_{\text{in}}^2$		○ $1 : d_{\text{out}}^2$	
○ $d_{\text{in}} : 1$		○ $d_{\text{out}} : 1$	
○ $d_{\text{in}}^2 : 1$		○ $d_{\text{out}}^2 : 1$	

ii. One option for applying muP to learnable biases is to treat biases as matrices with a shape of $d_{\text{out}} \times 1$, i.e. thinking of their $d_{\text{in}} = 1$ as though they were weights multiplying a constant feature whose value is always 1. We can then apply the same muP learning-rate scaling rule derived in part (a) of this problem for the learnable biases, along with applying the muP scaling for the weights based on their shape. **What is the resulting rough proportion of contribution-from-bias : contribution-from-dense-weight-matrix** for the resulting updates?
Below, d_{in} refers to the d_{in} of the learnable dense weight matrix. The d_{out} is the same for both the biases and the weight matrix.

Select one:

☑ $1 : 1$	i.e. same contribution	○ $1 : 2$	i.e. Bias contributes half as much
○ $1 : d_{\text{in}}$		○ $1 : d_{\text{out}}$	
○ $1 : d_{\text{in}}^2$		○ $1 : d_{\text{out}}^2$	
○ $d_{\text{in}} : 1$		○ $d_{\text{out}} : 1$	
○ $d_{\text{in}}^2 : 1$		○ $d_{\text{out}}^2 : 1$	

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304994738

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

12. Comparing Sequence Model Architectures: Compute (20 pts)

In this question, you will compare three major sequence model architectures: classic Recurrent Neural Networks (RNNs), Transformers, and modern State Space Models (SSMs). Let T be the sequence length and d be the embedding dimension. For State Space Models, let N be the state-expansion — i.e. by what factor (typically a number like 64) the state is bigger than the embedding dimension.

Assume that the cost of a single floating-point operation (FLOP) is constant. **Fill in the missing cells of the below table of runtimes (in big-O notation), and provide a brief explanation in the corresponding section on the next page.** You can assume that $T \gg d \gg N$.

Task	RNNs	Transformers	SSMs (S4D style)
Training cost	$O(Td^2)$ due to both RNN forward pass and backprop using T recurrent updates using matrices with dimension d	**(a)** $O(T^2d)$	**(b)** $O(d^2T)$
Training parallelization	Poor due to inherently sequential computation due to the nonlinearities encountered along the time axis	Excellent (every token's attention computation can be fully-independent)	Good (FFT can be parallel with depth $\log T$, but not as good as self-attention because of how GPUs are optimized.).
Inference cost for autoregressive generation of a sequence of T tokens	**(c)** $O(Td^2)$	**(d)** $O(T^2d)$	$O(Td(d+N))$ for T steps of cost $O(d^2 + dN)$ due to an update of the state with dimension dN that breaks down into $O(d^2)$ to compute inputs/outputs and $O(dN)$ to do the independent scalar state updates. Under our assumptions, the cost simplifies to $O(Td^2)$.

Because this is a cheat-sheet free exam, we provide you with some S4D basics here. S4D is the style you saw in lecture before Mamba-style models with selective state spaces.

- In modern SSMs, the state-space block performs a role analogous to that of a causal multi-head-attention block in Transformer architectures.
- In S4D models, the state-space part has diagonal learnable dynamics and the operation of the state-space block is to perform a different learnable linear-time-invariant scalar convolution for each embedding dimension.
- It is possible to speed up the computation of a full-length convolution of a scalar sequence of length n with another scalar sequence of length n to happen in $O(n \log n)$ time using FFT-based tricks.
- It is possible to directly materialize a single scalar impulse response of length T using $O(NT)$ computation but only $O(N + T)$ memory.
- As is the case in multi-head-attention for computing the queries, keys, and values, $O(d^2T)$ computations are needed to compute all the inputs and outputs to S4D's core state-space-model. The counterparts to fully-connected feedforward MLPs also cost the same.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304,994738

Brief Explanations For Your Answers:

(a) (2pts)

The core of self-attention:

- Calculate Q, K, V: $O(Td^2)$
- Calculate the attention matrix QK^T: $O(T^2d)$
- Use the attention weight multiply V: $O(T^2d)$

$\Rightarrow O(T^2d + Td^2) \approx O(T^2d)$ $(T \gg d)$

(b) (2pts)

S4D: For each channel, do a convolution (length T)

Can FFT: Each channel $O(T \log T)$, d channels in total $\Rightarrow O(d \log T)$

Add input/output projection, MLP ... $\Rightarrow O(d^2T)$

$\rightarrow O(d^2T + dT\log T) \approx O(d^2T)$ $(d \gg \log T)$

(c) (2pts)

Each step, one $d \cdot d$ incursive update, matrix multiply, $O(d^2)$

T steps in total $\Rightarrow O(Td^2)$

(d) (2pts)

The t-th step attention needs to do dot multiply with the past t tokens:

$O(td)$ (multi) Sum up $t = 1, \ldots, T$: $O(T^2d)$

In addition, each step projection/MLP: $O(d^2) \Rightarrow O(Td^2)$

$\rightarrow O(T^2d + Td^2) \approx O(T^2d)$ $(T \gg d)$

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

13. Residual Connections with PreNorms (20 pts)

The deeper a neural network grows, the more likely it will suffer from either vanishing gradients or exploding gradients due to the multiplicative nature of the chain rule.

Fig 1 shows a simplified example of a residual block with layer normalization in the now default "pre-norm" style.

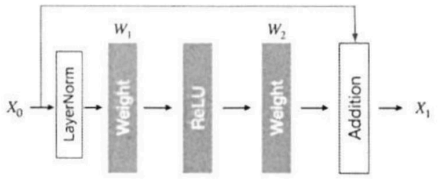


Figure 1: Diagram for a Residual Block with LayerNorm

(a) (3pts) Assuming that we have zero biases, **what is the expression for X_1 in terms of W_1, W_2 and X_0?** Assume $W_i \in \mathbb{R}^{n \times n}$, $X_i \in \mathbb{R}^n$. You can just call both LayerNorm and component-wise ReLU as functions here, you don't need to expand them.

$X_1 = X_0 + W_2\,\mathrm{ReLU}(W_1\,\mathrm{LN}(X_0))$

(b) (5pts) **Compute the Jacobian $\frac{\partial X_1}{\partial X_0}$.** Express your answer in terms of W_1, W_2, the Jacobian of LayerNorm, J, and the Jacobian of the ReLU given by diagonal matrix D whose i-th term on the diagonal is 1 if the corresponding ReLU is on, and 0 if it is off.

Based on the structure of your result, **explain why the residual connection helps prevent vanishing gradients during backprop for a deep network**.

Let $J = \frac{\partial \mathrm{LN}(X_0)}{\partial X_0}$ (Jacobian of LayerNorm)

Jacobian of ReLU is a diagonal matrix D.

where $D_n = 1$ if this dimension ReLU activation > 0, and otherwise 0

Chain Rule: $\frac{\partial X_1}{\partial X_0} = I + W_2 D W_1 J$ $\quad \frac{\partial X_1}{\partial X_0} = I + W_2 D W_1 J$

Explain: in backprop, the gradients will multiply many Jacobians of different layers. Even if the spactual norm of $W_2 D W_1 J$ may < 1, there will be always one term "I" in Jacobian that offers the direct way, in order to let the gradient go back through in a nearly equal way, instead of being crushed to 0 by the multiplies.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 304194738

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

(c) (12pts) For simplicity, instead of LayerNorm, let us consider an RMSNorm layer with no learnable parameters that scales its input vector so that the output vector points in the same direction, but has RMSNorm 1.

If the input is the $d-$dimensional vector $\mathbf{X}$, the output vector $\mathbf{Z} = \sqrt{d}\frac{\mathbf{X}}{\|\mathbf{X}\|_2}$.

When backpropping the gradient through the RMSNorm Layer, what is the applied Jacobian $J = \frac{\partial Z}{\partial X}$?

(Hint: while it is completely possible to grind the calculation out using vector calculus, it might be useful to think about this Jacobian J using what the derivative means conceptually. Let $\mathbf{R}(\mathbf{X})$ be the RMSNorm Layer acting on $\mathbf{X}$. Then $\mathbf{R}(\mathbf{X}+\boldsymbol{\Delta}\mathbf{X}) \approx \mathbf{R}(\mathbf{X}) + J\boldsymbol{\Delta}\mathbf{X} + O(\|\boldsymbol{\Delta}\mathbf{X}\|^2)$.

To understand J, consider a neighborhood of $\mathbf{X}$ using a local orthogonal coordinate system. In other words, use a convenient coordinate system for $\boldsymbol{\Delta}\mathbf{X}$. Let the first basis vector be aligned to $\mathbf{X}$ itself and have the rest be $\mathbf{h}$ that are orthogonal to it. Use the linearity of the derivative to understand those different directions separately.

What is the impact on the output of an RMSNorm Layer of growing/shrinking $\mathbf{X}$ by adding something aligned to $\mathbf{X}$?

What is the impact on the output of an RMSNorm Layer of adding a tiny $\mathbf{h}$ that is orthogonal to $\mathbf{X}$? — You might find it helpful to think about the Pythagorean theorem and the fact that $\sqrt{1+\Delta^2} \approx 1 + \frac{\Delta^2}{2}$ when Δ is small.

Once you understand what the matrix J does to all basis vectors, what must it be?)

Vector Calculus Facts from cheatsheat:

Derivative of a norm-squared: $\frac{\partial}{\partial \mathbf{x}}\mathbf{x}^\top\mathbf{x} = 2\mathbf{x}^\top$

Scalar-vector product rule: $\frac{\partial}{\partial \mathbf{x}}(f(\mathbf{x})\mathbf{g}(\mathbf{x})) = f(\mathbf{x})\frac{\partial}{\partial \mathbf{x}}(\mathbf{g}(\mathbf{x})) + \mathbf{g}(\mathbf{x})\frac{\partial}{\partial \mathbf{x}}(f(\mathbf{x}))$

This plus scalar calculus should suffice to get the answer.

Let $r = \|X\|_2$ $\quad \therefore Z = \sqrt{d}\, r^{-1} X$

$dZ = \sqrt{d}\left(\frac{1}{r}dX - \frac{1}{r^3}X(X^\top dX)\right)$ $\quad \therefore J = \frac{\sqrt{d}}{\|X\|_2}\left(I - \frac{XX^\top}{\|X\|_2^2}\right)$

Along the directions of X (i.e. only making it larger/smaller),
the outputs will be nearly the same.
$\Rightarrow$ Derivative will be 0 in that directions

In the orthogonal directions of X, the outputs will be scaled $\frac{\sqrt{d}}{\|X\|_2}$
and be projected to the orthogonal subspace.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3061994738

14. Deja Vu: Denoising diffusion models for generation (28 pts)

In lecture, we talked about denoising diffusion models to get samples from a continuous distribution. This problem is about the potentially simpler binary case. We will assume that we have an unknown distribution of black-and-white images $P(\mathbf{x})$ together with a very large number of example images $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$. Formally, each image can be viewed as a binary vector of length m, i.e. $\mathbf{x} \in \{-1, +1\}^m$.

The first conceptual step in setting up a diffusion model is to choose the easy-to-sample distribution that we want to have at the end of the forward diffusion. For this, we choose m iid fair coin tosses (here we think of a fair coin as having a 50% chance of being +1 and a 50% chance of being -1) arranged into a vector.

Next, we need to choose a way to incrementally degrade the images. Let $\mathbf{Y}_0$ start with whatever image sample $\mathbf{x}$ we want to start with. At diffusion stage t, we generate $\mathbf{Y}_t$ from $\mathbf{Y}_{t-1}$ by randomly flipping each pixel of $\mathbf{Y}_{t-1}$ independently with probability δ where δ is a small positive number.

It turns out that this process of rare pixel-flipping can be reinterpreted for easier analysis. For the j-th pixel at diffusion stage t, this process can alternatively be viewed as first flipping an independent coin $R_t[j]$ with a probability 2δ of coming up $+1$ and then, if $R_t[j] = +1$ replacing $Y_{t-1}[j]$ with a freshly drawn independent fair coin $F_t[j]$ that is equally likely to be -1 or $+1$. If $R_t[j] \neq +1$, we leave that pixel alone i.e. $Y_t[j] = Y_{t-1}[j]$.

(a) (4pts) We need to verify that if we do this and diffuse for sufficiently many stages T, that the resulting distribution is close to looking like m i.i.d. fair coins. **Show that the probability that pixel j has been replaced at some point by an independent fair coin by time T goes to 1 as $T \to \infty$.**
(HINT: It might be helpful to look at the probability that the desired event has not happened...)

Let E_T = "Until time T, there is at least one replacement"

$\therefore \overline{E_T}$ = "None replacement occurs in the first T stages"

$\therefore$ The probability of "each step, no replacement" is $1-2\delta$, and by independence:

$P(\overline{E_T}) = (1-2\delta)^T \quad \therefore P(E_T) = 1-(1-2\delta)^T \xrightarrow[T\to\infty]{} 1$

(as long as $0<2\delta<1$, i.e. $\delta \in (0, \frac{1}{2})$)

(b) (6pts) To efficiently do diffusion training, we need a way to be able to quickly sample a realization of $\mathbf{Y}_t$ starting from $\mathbf{Y}_0 = \mathbf{x}_i$. **Give a procedure to sample a realization of $\mathbf{Y}_t$ given $\mathbf{Y}_0$ without having to generate $\mathbf{Y}_1, \mathbf{Y}_2, \ldots, \mathbf{Y}_{t-1}$.**
(HINT: Your answer should involve flipping at most two (potentially biased) coins for each pixel.)

$P_{keep} = (1-2\delta)^t$ < if no replacement: $Y_t[j] = Y_0[j]$
if at least one replacement: last replacement turns it to a fair coin, i.e. $P(Y_t[j]) = 1$ or -1, $\frac{1}{2}$ each

For each j: toss a biased coin, $B \sim \text{Bernoulli}(1-P_{keep}) = \text{Bernoulli}\left(1-(1-2\delta)^t\right)$

if $B=0$: output $Y_t[j] = Y_0[j]$

if $B=1$: then toss a fair coin $F \in \{\pm 1\}$ ($\frac{1}{2}$ respectively), output $Y_t[j] = F$

 All Rights Reserved. This may not be publicly shared without explicit permission.

(c) (2pts) For the reverse diffusion process that will be used during image generation and is being learned during training, our goal is to approximate $P(\mathbf{Y}_{t-1}|\mathbf{Y}_t)$ with a neural net that has learnable parameters θ. Suppose I give you a neural net whose input is a binary image $\mathbf{Y}_t$ and whose output is m real numbers that could each in principle be from $-\infty$ to $+\infty$ (for example, these could be the outputs of a linear layer). **Which of the following nonlinear activation functions would be most appropriate to convert a real number into a probability that we could use to sample whether the pixel in question should be a $+1$?**

- [x] Sigmoid $\frac{1}{1+\exp(-x)}$
- [ ] ReLU $\max(0, x)$
- [ ] Tanh $\tanh(x) = \frac{\exp(2x)-1}{\exp(2x)+1}$

(d) (10pts) The goal of training is to approximate a probability distribution for random denoising. However, we do not actually have access to $P(\mathbf{Y}_{t-1}|\mathbf{Y}_t)$ and decide to use $P(\mathbf{Y}_{t-1}|\mathbf{Y}_t, \mathbf{Y}_0)$ instead as a proxy. Note: this can be interpreted as finding an appropriate target distribution that corresponds to trying to predict $\mathbf{Y}_0$ itself at this stage.

What is $P(Y_{t-1}[j] = +1|\mathbf{Y}_t = \mathbf{y}, \mathbf{Y}_0 = \mathbf{x})$?

For simplicity, just do this calculation for the case $x[j] = +1$. To further help you save some time, you may use the following helper result that comes from Bayes' Rule. If A and B are both binary random variables where the prior probabilities are $P(A = +1) = \rho$ and $P(A = -1) = 1 - \rho$, with B being bit-flipped from A with independent probability δ — that is, $P(B = +1|A = +1) = 1 - \delta$, $P(B = -1|A = +1) = \delta$, $P(B = +1|A = -1) = \delta$, and $P(B = -1|A = -1) = 1 - \delta$ — then the conditional probabilities for A conditioned on B are given by:

$$P(A = +1|B = +1) = \frac{(1-\delta)\rho}{(1-\rho)\delta + (1-\delta)\rho} \tag{10}$$

$$P(A = -1|B = +1) = \frac{(1-\rho)\delta}{(1-\rho)\delta + (1-\delta)\rho} \tag{11}$$

$$P(A = +1|B = -1) = \frac{\delta\rho}{\rho\delta + (1-\delta)(1-\rho)} \tag{12}$$

$$P(A = -1|B = -1) = \frac{(1-\delta)(1-\rho)}{\rho\delta + (1-\delta)(1-\rho)} \tag{13}$$

(HINT: What is the distribution for $Y_{t-1}[j]$ given $Y_0[j]$?)

From $Y_0[j]=+1$, each step change the sign by probability δ $\therefore P(Y_n=+1|Y_0=+1) = \frac{1}{2} + \frac{1}{2}(1-2\delta)^n$

Take $n=t-1$: $\rho = \frac{1}{2} + \frac{1}{2}(1-2\delta)^{t-1}$ Denote $b = y[j] \in \{\pm 1\}$

If $y[j]=+1$: $P(Y_{t-1}[j]=+1 \mid Y_t[j]=+1, Y_0[j]=+1) = \frac{(1-\delta)\rho}{(1-\rho)\delta + (1-\delta)\rho}$, where $\rho = \frac{1}{2} + \frac{1}{2}(1-2\delta)^{t-1}$

If $y[j]=-1$: $P(Y_{t-1}[j]=+1 \mid Y_t[j]=-1, Y_0[j]=+1) = \frac{\delta\rho}{\rho\delta + (1-\delta)(1-\rho)}$

This is what the problem asked (under the situation $X_{[j]}=+1$, divide two case by $y[j]$)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

(e) (6pts) Let the (conditional) probability distribution (on whether each pixel is $+1$) output by our neural net with nonlinearity be $Q_t(\mathbf{Y}_t)$. For training the denoising diffusion model $q(\mathbf{Y}_{t-1}|\mathbf{Y}_t)$, we choose to use SGD loss $D_{KL}(P(\mathbf{Y}_{t-1}|\mathbf{Y}_t, \mathbf{Y}_0 = \mathbf{x}_i)||Q_t(\mathbf{Y}_t))$ where $\mathbf{x}_i$ is the random training image drawn, t is the random time drawn from 1 to T, and $\mathbf{Y}_t$ is the randomly sampled realization of the forward diffusion at time t starting with the image $\mathbf{x}_i$ at time 0. This ends up being a loss on the vector of probabilities coming out of $Q_t(\mathbf{Y}_t)$ that can be written as a sum over the m entries in the vector of probabilities.

Given what you know about KL Divergence, what does this loss penalize most strongly? What does this loss look like at $t = 1$ in particular?

Because there are no cheat-sheets allowed and the order matters, we give you the formula for KL Divergence in the discrete case here: $D_{KL}(P(z)||Q(z)) = \sum_z P(z) \ln \frac{P(z)}{Q(z)}$.

For discrete variables: $D_{KL}(P\|Q) = \sum_z P(z) \ln \frac{P(z)}{Q(z)} = -\underbrace{\sum_z P(z) \ln Q(z)}_{\text{cross-entropy}} - H(p)$

where $H(p)$ is independent with Q

$\therefore$ minimize KL $\Longleftrightarrow$ minimize $E_{z\sim P}[-\ln Q(z)]$

$\therefore$ Its strongest penalty is: At the case that P thinks will most probably occur, Q gives a very small probability however, especially near 0, because $-\ln Q(z)$ will be exploding big!

At $t=1$, $t-1=0$ $\therefore Y_0$ is what we need to focast.

However, in the loss, $Y_0 = X_i$ is conditioned

$\therefore$ The distribution $P(Y_0|Y_1, Y_0=X_i)$ is actually a weakened distribution (At $Y_0 = X_i$, the probability is 1)

$\therefore$ KL turns to "the minus log likelihood to the true value", which is the pixel-focused Bernoulli cross entropy:

Assume the network outputs $q_j = Q_t(Y_t) P(Y_{t-1}[j] = +1 \,|\, Y_t)$ to each pixel j

$\therefore$ At $t=1$, the loss of each pixel is: $\ell_j = \begin{cases} -\ln q_j, & X_i[j] = +1 \\ -\ln(1-q_j), & X_i[j] = -1 \end{cases}$

The total loss is $\sum_{j=1}^{m} \ell_j$

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: Jie (Menger) Wen 3041994738

[Doodle page! Draw us something if you want or give us suggestions or complaints. You can also use this page to report anything suspicious that you might have noticed.

If needed, you can also use this space to work on problems. But if you want the work on this page to be graded, make sure you tell us on the problem's main page.]

I like this course very much ...
As a visiting student (major in quant) from China,
this is the first time that taking such a hard-core
CS course ... Really torturing TBH though
I spent at least half of each week to learn this course.
Actually I do not have very good math & CS basis originally
But I've tried my best. 
And this is the most cutting-edge course I've ever taken.
There are even lecture materials just published months before!
Fame has it CS courses of UC Berkeley are renowned
Now I beleive it.
Please be mercy  plz... thx...

(Oh.. Professors and GSIs are really nice and knowledgable and experienced
in deep learning ... I gained a lot from you guys! THX!!)

 All Rights Reserved. This may not be publicly shared without explicit permission.